

**Système de gestion des produits basé
sur la lecture de codes-barres**
[Cas de POS Librairie]
« LibrairyPro »

Réalisées par :

**KNIRI Doha
EL-BAZ Wissal**

Encadrée par :

Pr. ELAZZABY Fouzia

Soutenu le 20 juin 2026 Devant le jury

**Pr. ELAZZABY Fouzia
Mme BENAMRI Ichrak
Pr. SEKHARRA Youssef**

**Encadrante
Présidente
Examineur**

Avant-propos

Dans le cadre de notre formation en Développement Informatique à la Faculté des Sciences Ben M'Sick, nous avons été amenés à réaliser un Projet de Fin d'Études (PFE) afin de mettre en pratique les connaissances théoriques et techniques acquises tout au long de notre cursus universitaire.

Le présent projet, intitulé **LibrairyPro**, consiste en la conception et le développement d'une application web de gestion de point de vente (POS) destinée à une librairie moderne. Ce choix s'inscrit dans un contexte où la transformation numérique occupe une place de plus en plus importante dans les activités commerciales. Les commerces de détail sont aujourd'hui confrontés à de nombreux défis liés à la gestion des ventes, au suivi des stocks, à l'organisation des produits et à l'amélioration de la qualité du service offert aux clients. L'informatisation de ces processus constitue ainsi une solution efficace pour optimiser les opérations quotidiennes et réduire les risques d'erreurs.

L'objectif principal de LibrairyPro est de fournir une plateforme centralisée permettant d'automatiser et de simplifier les différentes tâches de gestion d'une librairie. L'application offre plusieurs fonctionnalités essentielles telles que l'enregistrement des produits, la gestion des catégories, le suivi des stocks, la réalisation des ventes, la génération de tickets de caisse ainsi que la consultation des statistiques de vente. Afin d'améliorer l'efficacité du système, une fonctionnalité de lecture de codes-barres a également été intégrée, permettant une identification rapide et fiable des produits lors des opérations de vente ou de gestion des stocks.

Par ailleurs, ce projet met en œuvre plusieurs technologies modernes de développement web afin de garantir une application performante, sécurisée et évolutive. L'architecture adoptée permet une gestion efficace des données ainsi qu'une interaction fluide entre les différents composants du système. Une attention particulière a également été accordée à l'expérience utilisateur grâce à la conception d'une interface intuitive, ergonomique et adaptée aux besoins des utilisateurs.

Au-delà de son aspect technique, ce projet nous a permis de développer de nombreuses compétences pratiques telles que l'analyse des besoins, la modélisation du système, la conception de bases de données, le développement d'applications web, les tests logiciels ainsi que la gestion de projet. Il constitue ainsi une expérience enrichissante qui nous a

permis de consolider nos acquis académiques tout en nous préparant aux exigences du monde professionnel.

Ce rapport présente l'ensemble des étapes ayant conduit à la réalisation de ce projet, depuis l'étude du contexte et l'analyse des besoins jusqu'à la conception, le développement, les tests et l'évaluation de la solution proposée.

Remerciements

Au terme de ce projet de fin d'études, nous souhaitons exprimer notre sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation.

Nous adressons tout d'abord nos plus vifs remerciements à notre encadrante, **Pr. Elazzaby Fouzia**, pour sa disponibilité, son accompagnement constant, ses précieux conseils ainsi que ses orientations pertinentes tout au long de ce travail. Son expertise, sa rigueur scientifique et son soutien nous ont permis de mener ce projet dans les meilleures conditions et d'enrichir nos connaissances tant sur le plan académique que professionnel.

Nous tenons également à remercier l'ensemble du corps professoral de la **Faculté des Sciences Ben M'Sick**, dont les enseignements et les compétences transmises au cours de notre formation ont constitué une base solide pour la réalisation de ce projet. Les connaissances acquises durant notre parcours universitaire ont joué un rôle essentiel dans la compréhension et la mise en œuvre des différentes étapes de ce travail.

Nos remerciements s'adressent aussi à l'administration de la **Faculté des Sciences Ben M'Sick** pour les ressources pédagogiques, techniques et documentaires mises à notre disposition, ainsi que pour l'environnement académique favorable à l'apprentissage, à la recherche et au développement des compétences.

Enfin, nous adressons nos remerciements les plus sincères à nos familles et à nos proches pour leur soutien moral, leur patience et leur confiance tout au long de notre parcours universitaire. Leur encouragement permanent a été une source de motivation essentielle pour mener à bien ce travail.

À toutes celles et ceux qui ont contribué, directement ou indirectement, à l'aboutissement de ce projet, nous adressons nos plus sincères remerciements.

Résumé

Ce projet de fin d'études présente la conception et le développement d'une application web de gestion de point de vente **POS** nommée **LibrairyPro**, destinée à une **librairie**. L'objectif principal est d'optimiser la gestion des produits, des ventes et des stocks grâce à un système intelligent basé sur la lecture de codes-barres.

L'application est développée avec **React + Vite** pour le frontend et **Spring Boot** pour le backend, avec une base de données **MySQL** pour les données. Le système est sécurisé par une authentification **JWT** avec gestion des rôles administrateur et caissier.

La solution propose un tableau de bord analytique avancé, une gestion complète des produits et fournisseurs, un système de caisse intelligent avec génération automatique de factures PDF et lecture de **codes-barres**.

Les résultats obtenus montrent une amélioration significative de la rapidité de traitement des ventes et une meilleure gestion des stocks.

✚ **Mots-clés** : POS, Code-barres, React, Vite, JWT, Spring Boot, MySQL, Librairie

Abstract

This end-of-study project presents the design and development of a point-of-sale **POS** web application named **LibrairyPro**, intended for a **bookstore**. The main objective is to optimize the management of products, sales, and inventory through an intelligent system based on barcode scanning.

The application is developed using **React + Vite** for the frontend and **Spring Boot** for the backend, with a **MySQL** database for data storage. The system is secured by **JWT** authentication with administrator and cashier role management.

The solution offers an advanced analytical dashboard, complete management of products and suppliers, an intelligent checkout system with automatic PDF invoice generation and **barcode** scanning.

The obtained results show a significant improvement in sales processing speed and better inventory management.

 **Keywords:** POS, Barcode, React,Vite, JWT, Spring Boot, MySQL, Bookstore

Glossaire

Terme	Définition
POS	Point Of Sale — Système informatique de gestion des ventes et des paiements au point de vente physique.
JWT	JSON Web Token — Standard ouvert (RFC 7519) permettant de sécuriser les échanges entre client et serveur via des tokens signés numériquement.
API	Application Programming Interface — Interface permettant à deux applications de communiquer entre elles via des requêtes standardisées.
REST	Representational State Transfer — Style d'architecture pour les API web basé sur des requêtes HTTP (GET, POST, PUT, DELETE) et des ressources identifiées par des URL.
CRUD	Create, Read, Update, Delete — Les quatre opérations de base pour la gestion des données en base de données.
MVC	Modèle-Vue-Contrôleur — Patron d'architecture logicielle séparant la logique métier (Modèle), l'affichage (Vue) et les interactions utilisateur (Contrôleur).
JPA	Java Persistence API — Spécification Java pour la persistance des données et le mapping objet-relationnel (ORM).
ORM	Object-Relational Mapping — Technique de programmation permettant de faire correspondre des objets Java avec des tables de base de données relationnelles.
Spring Security	Module du framework Spring Boot dédié à la sécurisation des applications Java (authentification, autorisation, protection CSRF, etc.).

Vite	Outil de build frontend moderne pour React/Vue offrant un démarrage rapide et un rechargement à chaud (Hot Module Replacement).
React Router	Bibliothèque de gestion des routes (navigation) dans les applications React, permettant le routage côté client (SPA).
KPI	Key Performance Indicator — Indicateur clé de performance permettant de mesurer et d'évaluer l'efficacité d'une activité ou d'un processus.
SPA	Single Page Application — Application web dont le contenu est chargé dynamiquement sans rechargement complet de la page.
JPA/Hibernate	Hibernate est l'implémentation la plus populaire de JPA, utilisée pour mapper les entités Java vers les tables de la base de données MySQL.

Liste des figures

Figure 1. 1 : Schéma conceptuel de la solution proposée	19
Figure 1. 2 Diagramme de Gantt prévisionnel.....	22
Figure 1. 3 Diagramme de Gantt réel.....	23
Figure 1. 4 Récapitulatif.....	23
Figure 2. 1 Schéma d'un système POS classique	26
Figure 2. 2 Exemple de code-barres.....	27
Figure 2. 3 Processus d'authentification JWT.....	28
Figure 2. 4 Évolution des systèmes POS dans le temps.....	29
Figure 2. 5 Classification des systèmes POS	30
Figure 2. 6 Interfaces de solutions POS existantes	31
Figure 3. 1 Architecture Frontend React + Vite	35
Figure 3. 2 Architecture backend Spring Boot.....	37
Figure 3. 3 Structure de la base de données	38
Figure 3. 4 Diagramme de cas d'utilisation	39
Figure 3. 5 Diagramme de classes	40
Figure 3. 6 Diagramme de séquence authentification	41
Figure 3. 7 Diagramme de séquence vente avec scan du code barre	42
Figure 3. 8 Diagramme de séquence d'ajout de produit avec scan de code-barres	43
Figure 4. 1 Technologies utilisées Frontend.....	47
Figure 4. 2 Technologies utilisées Backend	48
Figure 4. 3 Outils de développement.....	50
Figure 4. 4 Page initiale.....	51
Figure 4. 5 Page d'accueil du caissier permettant la gestion des produits et la lecture des code-barres [scan code-barre off]	51
Figure 4. 6 Page d'accueil du caissier permettant la gestion des produits et la lecture des code-barres [scan code-barre on].....	52
Figure 4. 7 Page de panier	52
Figure 4. 8 Alerte de validation de panier/vente.....	53
Figure 4. 9 Page du facture	53
Figure 4. 10 Facture en PDF	54
Figure 4. 11 Page d'accueil du dashboard admin 1	55

Figure 4. 12 Page d'accueil du dashboard admin 2	55
Figure 4. 13 Page d'accueil du dashboard admin 3 [Alerte du stock]	56
Figure 4. 14 Page de stock/produit du dashboard admin 1.....	56
Figure 4. 15 Page de stock/produit [ajout du produit] du dashboard admin 2.....	57
Figure 4. 16 Page de fournisseur du dashboard admin 1	57
Figure 4. 17 Page de fournisseur [ajout du fournisseur] du dashboard admin 2	58
Figure 4. 18 Page du caissier du dashboard admin 1.....	58
Figure 4. 19 Page du caissier [ajout du caissier] du dashboard admin 2	59
Figure 4. 20 Page de la vente [avec test du recherche et vue de détails] du dashboard admin	59
Figure 4. 21 Page d'historique des ventes du dashboard admin	60
Figure 4. 22 Page du paramètre du dashboard admin	60
Figure Erreur 1 Mot de passe incorrect	81
Figure Erreur 2 Fausse Forme d'e-mail	81
Figure Erreur 3 Insertion incomplète	82
Figure Erreur 4 Produit introuvable [code-barre incorrecte]	82
Figure Erreur 5 Produit déjà existant [au cours d'ajout d'un produit par l'admin]	83
Figure Erreur 6 Forme de téléphone non valide	83

Liste des tableaux

Table 1 Technologie et justification du choix	20
Table 2 Comparaison des solutions POS	32
Table 3 Description de la base de données MySQL.....	49
Table 4 Tests fonctionnels [positifs (flux nominaux)]	68
Table 5 Tests fonctionnels [négatifs (cas d'erreur)]	69
Table 6 Métriques de performance	70

Table des matières

Avant-propos.....	1
Remerciements	3
Résumé.....	4
Abstract.....	5
Glossaire.....	6
Liste des figures.....	8
Liste des tableaux.....	10
Table des matières	11
Introduction	15
CHAPITRE 1 CONTEXTE GÉNÉRAL DU PROJET	17
1. Introduction	18
2. Présentation de l'existant et problématique	18
Problèmes identifiés :.....	18
3. Solution proposée et objectifs.....	18
Objectifs :	18
4. Présentation du projet.....	19
5. Gestion du projet.....	21
Méthodologie adoptée.....	21
 Planification	22
Analyse comparative entre planification prévisionnelle et réelle	24
6. Conclusion du chapitre	24
CHAPITRE 2 ÉTAT DE L'ART	25
1. Introduction du chapitre.....	26
2. Définitions et concepts clés	26
 Système POS (Point Of Sale).....	26
 Code-barres.....	27
 Gestion de stock.....	27

✂ Authentification JWT	27
3. Historique et évolution des systèmes POS.....	28
4. Types de systèmes existants	29
1. POS traditionnels	29
2. POS cloud	29
3. POS intelligents.....	30
5. Revue des solutions existantes	30
✂ Square POS.....	30
✂ Shopify POS.....	31
✂ Lightspeed POS	31
6. Tableau comparatif des solutions	32
7. Limites des solutions existantes.....	32
8. Positionnement et originalité du projet.....	33
9. Conclusion du chapitre	33
CHAPITRE 3 ÉTUDE ET ANALYSE DE LA SOLUTION PROPOSÉE.....	34
1. Introduction du chapitre.....	35
2. Architecture générale du système	35
1. Couche Présentation (Frontend).....	35
2. Couche Métier (Backend)	36
3. Couche Données.....	38
3. Modélisation UML du système	39
3.1 Diagramme de cas d'utilisation.....	39
3.2 Diagramme de classes	40
3.3 Diagrammes de séquence.....	41
4. Flux de données entre composants	44
5. Système de code-barres	44
6. Génération de facture PDF	44
7. Authentification et sécurité (JWT)	45
8. Conclusion du chapitre	45
CHAPITRE 4 RÉALISATION	46

1. Introduction du chapitre.....	47
2. Outils et technologies utilisés	47
- Technologies Frontend	47
- Technologies Backend	48
- Bases de données.....	48
- Outils de développement	50
3. Présentation des interfaces de l'application	51
3.1 Système de caisse (POS)	51
3.2 Dashboard administrateur	55
4. Extraits de code	61
4.1 Entité JPA — Produit (Backend Spring Boot).....	61
4.2 Endpoint REST — Recherche par code-barres (Backend Spring Boot).....	62
4.3 Configuration JWT — Spring Security (Backend).....	62
4.4 Composant React — Scan de code-barres (Frontend)	63
5. Gestion du projet avec Git.....	65
6. Conclusion du chapitre	65
CHAPITRE 5 TESTS ET ÉVALUATION	66
1. Introduction du chapitre.....	67
2. Objectifs des tests	67
3. Méthodologie de test	67
4. Scénarios de test	68
 Tableau des tests fonctionnels	68
5. Métriques de performance.....	70
6. Limites identifiées.....	70
7. Conclusion du chapitre	70
CHAPITRE 6 BILAN ET PERSPECTIVES	71
1. Introduction du chapitre.....	72
2. Synthèse du projet	72
3. Bilan des objectifs.....	72
3.1 Objectifs pleinement atteints	72

3.2 Objectifs partiellement atteints	73
3.3 Objectifs non atteints	73
4. Compétences développées.....	73
Compétences techniques :	73
Compétences méthodologiques :	74
5. Difficultés rencontrées	74
6. Perspectives d'amélioration	74
1. Application mobile (React Native)	74
2. Mode hors ligne (Offline POS)	75
3. Intelligence artificielle	75
4. Extension multi-boutiques.....	75
5. Amélioration UX/UI	75
7. Conclusion du chapitre	76
CONCLUSION GÉNÉRALE	77
BIBLIOGRAPHIE	78
ANNEXE	79
Annexe A : Script SQL de création de la base de données (extrait)	79
Annexe B : Capture de quelques erreurs	81
Annexe C : Lien du dépôt GitHub.....	83

Introduction

Avec l'évolution rapide du commerce de détail et la transformation numérique des entreprises, les librairies modernes sont aujourd'hui confrontées à de nouveaux défis en matière de gestion et d'organisation. Elles doivent être capables de traiter un volume croissant d'informations tout en assurant une rapidité, une fiabilité et une efficacité optimales dans leurs opérations quotidiennes.

Les méthodes traditionnelles de gestion, souvent basées sur des outils manuels ou des systèmes peu automatisés, montrent leurs limites face aux exigences actuelles du marché. Elles peuvent entraîner des erreurs de saisie, une perte de temps dans le traitement des ventes, ainsi qu'une difficulté à suivre en temps réel l'état des stocks et les performances commerciales. Ces insuffisances peuvent impacter directement la productivité et la qualité du service offert aux clients.

Dans ce contexte, la mise en place de solutions informatiques modernes devient une nécessité afin d'optimiser les processus internes et d'améliorer la prise de décision.

C'est dans cette optique que nous avons développé **LibrairyPro**, une solution web intelligente de gestion de point de vente (POS) dédiée aux librairies. Ce système a été conçu pour répondre aux besoins concrets des commerçants en automatisant plusieurs tâches essentielles telles que la gestion des ventes, le suivi des stocks, ainsi que l'organisation des produits.

L'une des principales fonctionnalités de LibrairyPro repose sur l'intégration de la lecture de codes-barres, permettant une identification rapide et fiable des articles lors des opérations de vente et de gestion. Cette approche réduit considérablement les erreurs humaines et améliore la fluidité du processus de caisse.

En outre, la solution vise à offrir une meilleure visibilité sur l'activité commerciale grâce à une gestion centralisée des données et une mise à jour en temps réel des informations. Cela permet aux utilisateurs de prendre des décisions plus efficaces basées sur des données précises et actualisées.

Ainsi, ce projet s'inscrit pleinement dans la dynamique de digitalisation des systèmes de gestion et constitue une réponse adaptée aux besoins actuels des librairies modernes.

Ce rapport est organisé en six chapitres :

- Le Chapitre 1 présente le contexte général et la problématique ;
- Le Chapitre 2 dresse un état de l'art des systèmes POS existants ;
- Le Chapitre 3 analyse l'architecture et la conception de la solution ;
- Le Chapitre 4 décrit la réalisation technique ;
- Le Chapitre 5 présente les tests et l'évaluation ;
- Le Chapitre 6 dresse le bilan et les perspectives d'amélioration.

CHAPITRE 1

CONTEXTE GÉNÉRAL DU PROJET



1. Introduction

Ce chapitre a pour objectif de présenter le cadre général dans lequel s'inscrit le projet LibrairyPro. Nous commençons par analyser la situation existante dans les librairies traditionnelles et les problèmes qui en découlent, puis nous présentons la solution proposée, ses objectifs fonctionnels, ainsi que la méthodologie de gestion de projet adoptée pour mener à bien ce travail.

2. Présentation de l'existant et problématique

Dans de nombreuses librairies, la gestion des produits et des ventes est encore effectuée manuellement ou à l'aide de systèmes basiques.

Problèmes identifiés :

- Saisie manuelle des produits → erreurs fréquentes
- Difficulté de suivi des stocks en temps réel
- Absence d'analyse des ventes
- Processus de caisse lent
- Manque d'intégration entre vente et stock

✚ Ces limites impactent directement la performance commerciale.

3. Solution proposée et objectifs

Le système **LibrairyPro** propose une solution complète :

Objectifs :

- Automatiser la gestion des ventes via code-barres
- Centraliser les données produits et clients
- Fournir un tableau de bord analytique

- Optimiser la gestion de stock en temps réel
- Générer des factures PDF automatiquement
- Sécuriser l'accès via authentification JWT

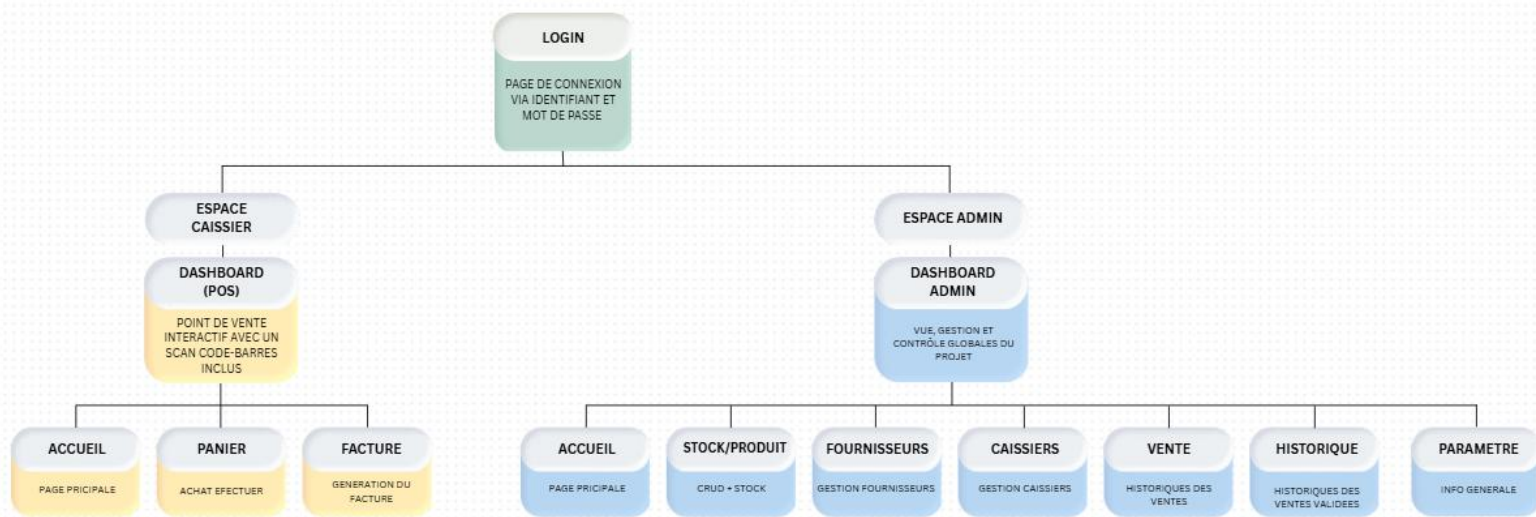


Figure 1. 1 : Schéma conceptuel de la solution proposée

4. Présentation du projet

LibrairyPro est une application web POS composée de deux interfaces :

- **Interface administrateur** : gestion globale du système
- **Interface caissier** : gestion des ventes en temps réel

Technologies utilisées :

- Frontend : React + Vite
- Backend : Spring Boot
- Base de données : MySQL
- Authentification : JWT

Technologie	Choix retenu	Alternatives considérées	Justification du choix
Frontend	React + Vite	Angular, Vue.js	React offre une grande flexibilité et une vaste communauté. Vite accélère le build. Angular est plus lourd et nécessite TypeScript obligatoire. Vue.js, bien que plus simple, a un écosystème plus limité pour des applications POS complexes.
Backend	Spring Boot	Node.js (Express), Django	Spring Boot offre robustesse, sécurité (Spring Security), et ORM natif (JPA/Hibernate). Node.js est moins adapté pour des applications métier complexes avec transactions ACID. Django nécessite une connaissance Python.
Base de données	MySQL	PostgreSQL, MongoDB	MySQL est une base relationnelle mature, idéale pour des données structurées avec des relations fortes (produits, ventes, factures). PostgreSQL aurait été également valide. MongoDB (NoSQL) n'est pas adapté car les données du POS sont fortement relationnelles.
Authentification	JWT	Sessions HTTP, OAuth2	JWT permet une authentification stateless, idéale pour les API REST. Les sessions HTTP nécessitent un état côté serveur. OAuth2 est surdimensionné pour un système interne.
Communication	API REST	GraphQL, SOAP	REST est une architecture simple, standardisée et largement supportée par Spring Boot et les frameworks frontend modernes comme React. GraphQL ajoute de la complexité sans bénéfice notable pour ce projet.

Table 1 Technologie et justification du choix

5. Gestion du projet

Méthodologie adoptée

Pour la réalisation de ce projet, nous avons adopté une approche de développement incrémental et itératif. Cette méthode consiste à diviser le projet en plusieurs incréments fonctionnels indépendants, chacun apportant une valeur ajoutée immédiate et pouvant être testé séparément.

--- Ce choix est justifié par plusieurs raisons :

- 1) la nature du projet ne nécessitait pas de coordination avec une équipe cliente externe, ce qui a rendu une méthode Agile complète (avec sprints formalisés et rétrospectives) moins adaptée ;
- 2) l'approche incrémentale nous a permis de valider chaque module avant de passer au suivant, limitant les risques d'erreur de conception tardive ;
- 3) elle s'adapte bien au travail en binôme avec une répartition claire des tâches.

--- Les phases du développement ont été organisées comme suit :

- 1) Analyse des besoins et spécifications fonctionnelles,
- 2) Conception de l'architecture et de la base de données,
- 3) Développement du backend (API Spring Boot),
- 4) Développement du frontend (React),
- 5) Intégration et tests,
- 6) Corrections et finalisation.

Planification

DIAGRAMME DE GANTT PRÉVISIONNEL — LibraryPro PFE 2025-2026			
Phase	Tâche	Début	Fin
Planification	Choix du projet & définition des besoins	15/03/2026	21/03/2026
Conception	Analyse & modélisation UML	22/03/2026	28/03/2026
Conception	Création de la base de données MySQL	29/03/2026	04/04/2026
Développement	Développement Backend (Spring Boot + JWT)	05/04/2026	25/04/2026
Développement	Développement Frontend (React + Vite)	26/04/2026	16/05/2026
Intégration	Liaison Frontend ↔ Backend (API REST)	17/05/2026	23/05/2026
Finalisation	Insertion des données & tests	24/05/2026	30/05/2026
Rapport	Rédaction du rapport PFE	31/05/2026	07/06/2026
Présentation	Préparation PowerPoint	08/06/2026	17/06/2026

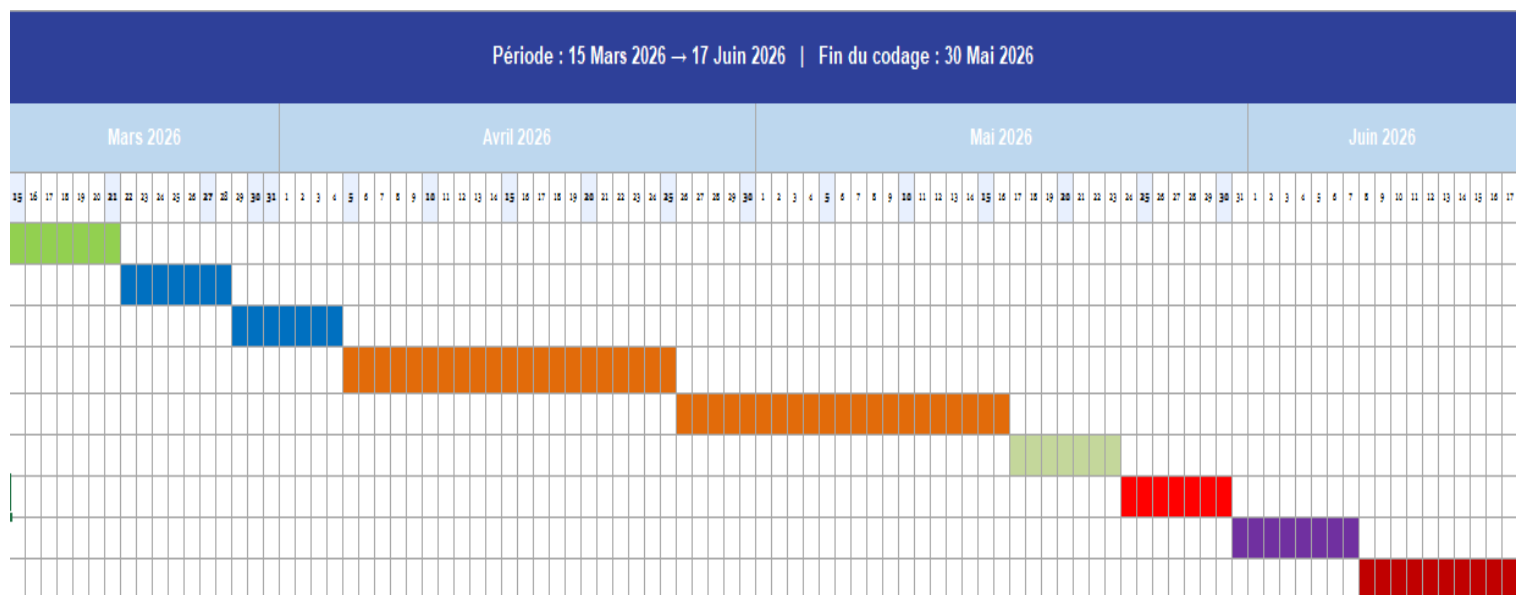


Figure 1. 2 Diagramme de Gantt prévisionnel

Diagramme de Gantt Réel — LibraryPro PFE 2025-2026			
Phase	Tâche	Réel Début	Réel Fin
Planification	Choix du projet & définition des besoins	15/03/2026	28/03/2026
Conception	Analyse & modélisation UML	29/03/2026	07/04/2026
Conception	Création de la base de données MySQL	08/04/2026	14/04/2026
Développement	Développement Backend (Spring Boot + JWT)	15/04/2026	05/05/2026
Développement	Développement Frontend (React + Vite)	06/05/2026	22/05/2026
Intégration	Liaison Frontend ↔ Backend (API REST)	23/05/2026	27/05/2026
Finalisation	Insertion des données & tests	28/05/2026	01/06/2026
Rapport	Rédaction du rapport PFE	02/06/2026	10/06/2026
Présentation	Préparation PowerPoint	11/06/2026	17/06/2026

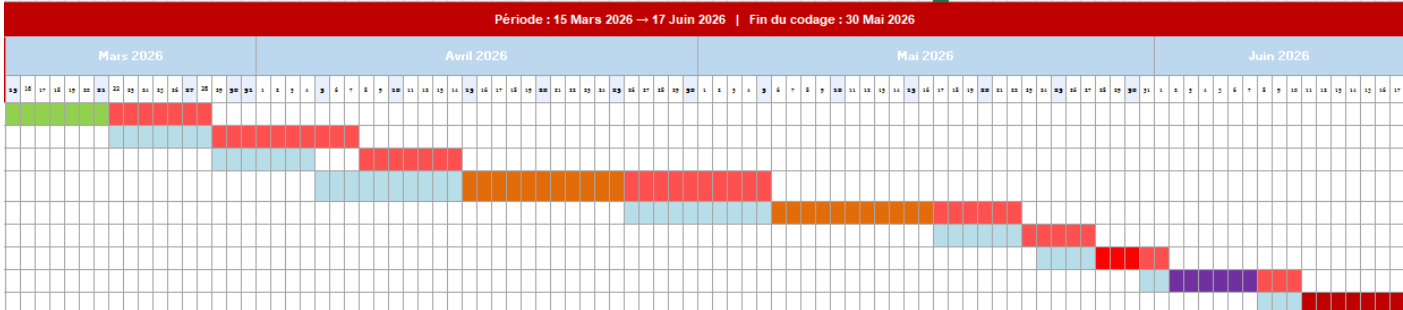


Figure 1. 3 Diagramme de Gantt réel

TABLEAU RÉCAPITULATIF — Comparaison Prévisionnel vs Réel							
Phase	Tâche	Prévu Début	Prévu Fin	Réel Début	Réel Fin	Écart	Remarque
Planification	Choix du projet & définition des besoins	15/03/2026	21/03/2026	15/03/2026	28/03/2026	+1 sem	Hésitation entre plusieurs sujets de PFE
Conception	Analyse & modélisation UML	22/03/2026	28/03/2026	29/03/2026	07/04/2026	+1 sem	Retard propagé depuis la planification
Conception	Création de la base de données MySQL	29/03/2026	04/04/2026	08/04/2026	14/04/2026	+4 j	Retard partiellement rattrapé
Développement	Développement Backend (Spring Boot + JWT)	05/04/2026	25/04/2026	15/04/2026	05/05/2026	+3 j	Difficulté config Spring Security + CORS
Développement	Développement Frontend (React + Vite)	26/04/2026	16/05/2026	06/05/2026	22/05/2026	+6 j	Compensé par maîtrise de React
Intégration	Liaison Frontend ↔ Backend (API REST)	17/05/2026	23/05/2026	23/05/2026	27/05/2026	-2 j	Intégration plus rapide que prévu
Finalisation	Insertion des données & tests	24/05/2026	30/05/2026	28/05/2026	01/06/2026	=	Dans les délais
Rapport	Rédaction du rapport PFE	31/05/2026	07/06/2026	02/06/2026	10/06/2026	+3 j	Corrections demandées par l'encadrante
Présentation	Préparation PowerPoint	08/06/2026	17/06/2026	11/06/2026	17/06/2026	=	Dans les délais

Figure 1. 4 Récapitulatif

Analyse comparative entre planification prévisionnelle et réelle

La comparaison entre le Gantt prévisionnel et le Gantt réel révèle plusieurs écarts. La phase de développement du backend a nécessité plus de temps que prévu, principalement en raison de la complexité de l'implémentation de la sécurité JWT et de la gestion des relations entre entités JPA. En revanche, la phase de développement frontend a été réalisée dans les délais grâce à la modularité offerte par React.

La phase d'intégration a également demandé un effort supplémentaire non planifié, dû aux problèmes de CORS entre le serveur Spring Boot et le client React, nécessitant une configuration spécifique de Spring Security. Ces délais ont été absorbés en réduisant le temps consacré à l'amélioration de l'interface utilisateur.

6. Conclusion du chapitre

Ce chapitre a permis de présenter le contexte général du projet, la problématique existante ainsi que la solution proposée. Le chapitre suivant sera consacré à l'état de l'art et à l'analyse des systèmes existants dans le domaine des points de vente.

CHAPITRE 2 ÉTAT DE L'ART



1. Introduction du chapitre

Les systèmes de gestion de point de vente (POS) ont connu une évolution importante avec la digitalisation du commerce de détail. Ils sont passés de simples logiciels de facturation à des plateformes complètes intégrant la gestion des stocks, l'analyse des ventes et l'automatisation des processus métiers.

Dans ce chapitre, nous présentons les concepts fondamentaux liés aux systèmes POS, une revue des solutions existantes, ainsi que le positionnement du projet **LibrairyPro** par rapport à ces solutions.

2. Définitions et concepts clés

✂ Système POS (Point Of Sale)

Un système POS (Point Of Sale) est une solution informatique permettant de gérer l'ensemble des transactions commerciales en un point de vente physique. Selon Laudon & Laudon [1], un POS moderne intègre non seulement la gestion des paiements, mais aussi la gestion des stocks, la fidélisation client, et l'analyse des ventes. Un POS se compose généralement d'un terminal (ordinateur, tablette ou registre enregistreur), d'un lecteur de codes-barres, d'une imprimante de tickets, et d'un logiciel de gestion.

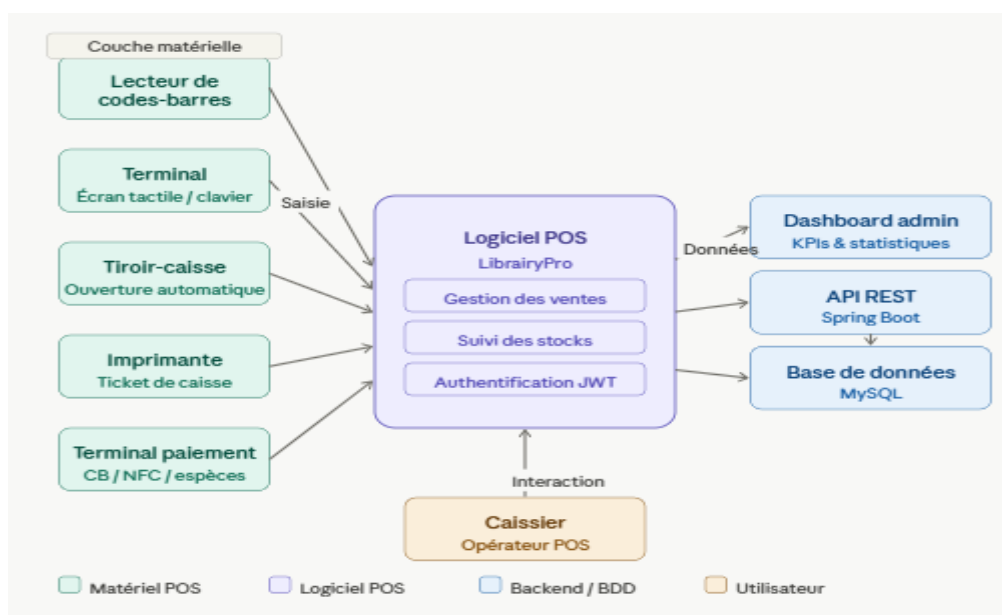


Figure 2. 1 Schéma d'un système POS classique

Code-barres

Un code-barres est une représentation graphique de données numériques permettant l'identification automatique et rapide d'un produit par un lecteur optique. Dans le commerce de détail, les codes-barres les plus utilisés sont le format EAN-13 (European Article Number, 13 chiffres) pour les produits de grande consommation, et l'ISBN (International Standard Book Number) pour les livres [2]. Le code-barres associé à chaque produit dans LibrairyPro suit le format EAN-13 et ISBN, conformément aux standards internationaux.



Figure 2. 2 Exemple de code-barres

Gestion de stock

La gestion de stock consiste à surveiller et contrôler en temps réel les entrées et sorties des produits afin d'éviter les ruptures de stock (pénuries) ou les surstocks (excédents coûteux). Les méthodes modernes de gestion de stock s'appuient sur des systèmes informatiques automatisés reliés au point de vente, permettant une mise à jour instantanée des quantités disponibles à chaque transaction [3].

Authentification JWT

JWT (JSON Web Token) est un standard ouvert défini par la RFC 7519 permettant de transmettre de manière sécurisée des informations entre deux parties sous forme d'objet JSON. Un token JWT est composé de trois parties encodées en Base64 et séparées par des points : l'en-tête (Header) contenant l'algorithme de signature (HS256), la charge utile (Payload) contenant les claims (informations de l'utilisateur, rôle, expiration), et la signature permettant de vérifier l'intégrité du token [4]. Dans LibrairyPro, le JWT est généré par le backend Spring Boot lors de la connexion et est envoyé dans l'en-tête HTTP Authorization de chaque requête API.

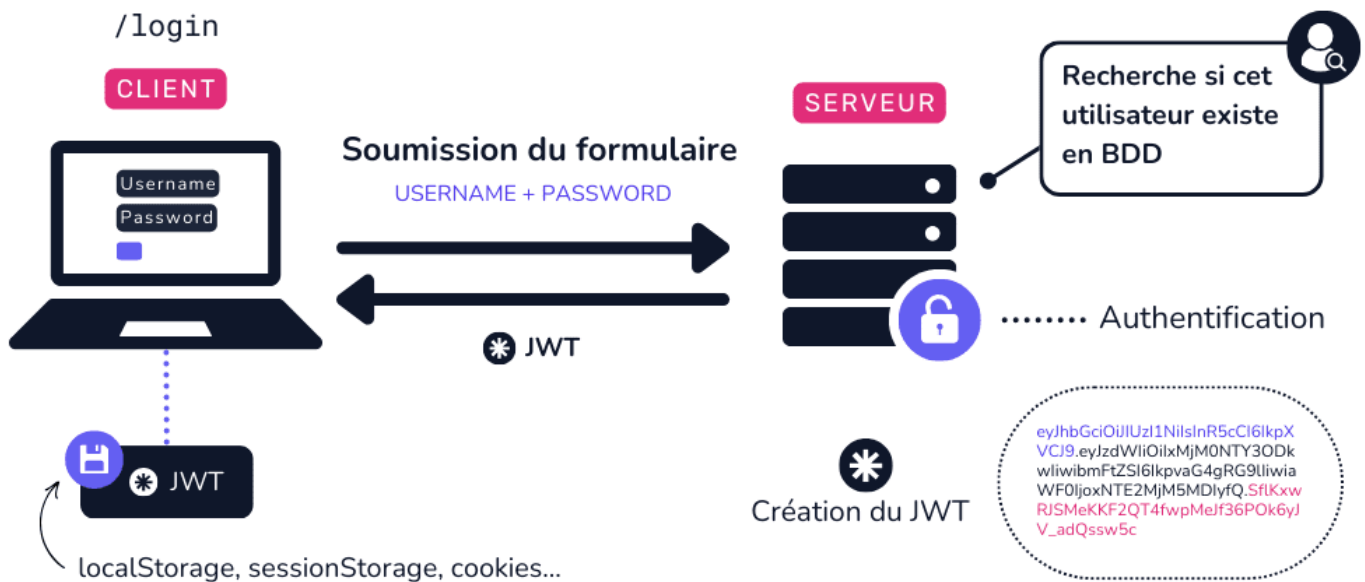


Figure 2. 3 Processus d'authentification JWT

3. Historique et évolution des systèmes POS

Les systèmes POS ont évolué en plusieurs étapes :

- 📁 Années 90 : systèmes locaux simples (facturation uniquement)
- 🌐 Années 2000 : logiciels connectés à des bases de données
- ☁ Années 2010 : apparition du cloud POS
- 🤖 Aujourd'hui : POS intelligents avec analytics et automatisation

Pour mieux comprendre : Les systèmes POS ont évolué en plusieurs phases depuis leur apparition dans les années 1970. Les premières caisses électroniques des années 1970-1980 se limitaient à l'enregistrement des ventes. Dans les années 1990, l'apparition des premiers logiciels POS sur PC a permis d'intégrer la gestion basique des stocks et la génération de rapports. Les années 2000 ont vu l'émergence de systèmes connectés à des bases de données relationnelles permettant une gestion multi-caisses. Depuis les années 2010, le cloud POS a révolutionné le secteur en permettant l'accès aux données en temps réel depuis n'importe

quel appareil connecté [5]. Aujourd'hui, les POS intelligents intègrent l'intelligence artificielle pour la prévision des ventes et la recommandation produits.



Figure 2. 4 Évolution des systèmes POS dans le temps

4. Types de systèmes existants

1. POS traditionnels

Les POS traditionnels fonctionnent en local sur l'infrastructure informatique du commerce. Ils offrent une indépendance vis-à-vis de la connectivité internet, mais présentent des contraintes importantes : coût de maintenance élevé, difficulté de mise à jour, et impossibilité d'accès aux données à distance.

2. POS cloud

Les systèmes POS cloud stockent leurs données sur des serveurs distants accessibles via internet. Ils permettent une synchronisation en temps réel entre plusieurs points de vente, des mises à jour automatiques, et un accès aux tableaux de bord depuis n'importe quel appareil. Leur principal inconvénient est la dépendance à une connexion internet stable.

3. POS intelligents

Les POS intelligents représentent la génération actuelle, combinant les avantages du cloud avec des fonctionnalités avancées d'analyse prédictive, d'intégration avec des plateformes e-commerce, et d'automatisation des processus métiers grâce à l'intelligence artificielle.

CLASSIFICATION DES SYSTÈMES POS

Comparatif des 3 grandes familles de Points de Vente

1. POS TRADITIONNELS (On-Premise)	2. POS CLOUD (SaaS / Moderne)	3. POS INTELLIGENTS (IA & Autonomes)
[Terminal lourd & serveur physique] • Stockage local : Données sauveées sur un serveur physique en arrière-boutique. • Matériel propriétaire : Écrans dédiés, câblages complexes et maintenance physique. • Dépendance réseau : Fonctionne sans internet, mais inaccessible à distance. • Coût initial élevé : Investissement lourd de départ en licences et serveurs. • Exemples : Anciennes caisses IBM, systèmes Micros ou installations locales.	[Tablette, accès web, synchro live] • Stockage distant : Ventes et stocks synchronisés en direct sur le Cloud. • Mobilité & Flexibilité : S'exécute sur iPad, tablettes Android ou navigateurs web. • Omnicanalité native : Fusion directe entre la boutique physique et l'e-commerce. • Modèle SaaS : Abonnement mensuel abordable avec mises à jour automatiques. • Exemples : Shopify POS, Lightspeed, Square, Toast.	[Vision par ordinateur & IA] • Automatisation par IA : Analyse prédictive et réapprovisionnement automatique. • Expérience sans caisse : Technologie 'Just Walk Out' via capteurs intelligents. • Reconnaissance avancée : Identification biométrique ou par caméra pour la fidélité. • Prix dynamiques : Ajustement en temps réel selon l'affluence ou la péremption. • Exemples : Magasins Amazon Go, reconnaissance de plateaux par caméra.

Figure 2. 5 Classification des systèmes POS

5. Revue des solutions existantes

Square POS

Square POS est une solution développée par Block, Inc. (anciennement Square, Inc.), lancée en 2009 et désormais utilisée par plus de 2 millions de commerçants dans le monde [6]. Elle est particulièrement adaptée aux petites et moyennes entreprises grâce à sa facilité d'utilisation et son modèle économique sans abonnement fixe (commission sur les transactions). Square propose une gestion des stocks basique, un suivi des ventes et une intégration avec diverses méthodes de paiement. Ses limites principales sont la personnalisation limitée et l'absence de fonctionnalités avancées pour des secteurs spécialisés comme les librairies.

Shopify POS

Shopify POS est l'extension physique de la plateforme e-commerce Shopify, permettant de synchroniser les ventes en ligne et en magasin [7]. Cette solution est particulièrement adaptée aux boutiques hybrides (physique + en ligne). Elle offre une gestion des stocks synchronisée, des rapports de vente détaillés, et une intégration native avec l'écosystème Shopify. En revanche, son coût d'abonnement élevé (à partir de 89\$/mois) et sa dépendance à l'écosystème Shopify représentent des contraintes importantes pour les petits commerces.

Lightspeed POS

Lightspeed POS est une solution cloud avancée spécialement conçue pour le commerce de détail, la restauration et les hôtels [8]. Elle propose une gestion de stock sophistiquée avec suivi des numéros de série, des analyses de vente avancées, et des intégrations avec de nombreuses plateformes tierces. Lightspeed est reconnue pour sa capacité à gérer des catalogues produits très étendus, ce qui en fait une solution adaptée aux librairies à grand volume. Son prix élevé (à partir de 69\$/mois) constitue néanmoins un frein pour les petits établissements.

	 Square SQUARE POS 	 shopify SHOPIFY POS 	 lightspeed LIGHTSPEED POS 
Philosophie	Philosophie UI (Interface): Minimaliste & Épurée	Philosophie UI (Interface): Modulable & Omnicanale	Philosophie UI (Interface): Densé & Métier (Expert)
Force	Encaissement ultra-rapide, idéal pour vendeurs saisonniers.	Vue unifiée client (Historique Web + Boutique).	Gestion avancée de stocks complexes (Variantes, Multi-boutiques).
Apprentissage	Courbe d'apprentissage : Immédiate (zéro formation).	Courbe d'apprentissage : Simple (pour utilisateurs Shopify).	Courbe d'apprentissage : Modérée à Difficile.
Cible	Target: Food trucks, cafés, artisans.	Target: Commerçants physiques avec e-commerce actif.	Target: Boutiques de mode/vélo, restaurants à fort volume.

Figure 2. 6 Interfaces de solutions POS existantes

6. Tableau comparatif des solutions

Critères	Square POS	Shopify POS	Lightspeed	LibrairyPro
Gestion stock	Moyenne	Bonne	Très bonne	★ Très avancée
Code-barres	Oui	Oui	Oui	★ Temps réel + SVG
Analytics	Basique	Moyen	Avancé	★ Dashboard KPI temps réel
Personnalisation	Faible	Moyenne	Moyenne	★ Très élevée
Mode offline	Oui	Partiel	Oui	En développement

Table 2 Comparaison des solutions POS

7. Limites des solutions existantes

Malgré leur efficacité, les solutions actuelles présentent plusieurs limites :

- Manque de personnalisation avancée
- Gestion de stock parfois non temps réel
- Coût élevé des solutions avancées
- Absence d'intégration mobile fluide pour scan avancé

8. Positionnement et originalité du projet

Le projet **LibrairyPro** se distingue par :

-  Lecture de code-barres intégrée dans le flux de vente
-  Dashboard analytique avancé en temps réel
-  Génération automatique de factures PDF
-  Architecture MySQL

9. Conclusion du chapitre

Ce chapitre a permis d'analyser les systèmes POS existants ainsi que leurs limites. Il met en évidence la nécessité d'une solution plus moderne, flexible et intelligente. Le projet **LibrairyPro** se positionne comme une réponse innovante combinant gestion de vente, analyse de données et automatisation avancée.

CHAPITRE 3 ÉTUDE ET ANALYSE DE LA SOLUTION PROPOSÉE



1. Introduction du chapitre

Ce chapitre présente l'analyse et la conception de la solution **LibrairyPro**. Il décrit l'architecture globale du système, les choix technologiques effectués ainsi que le flux de données entre les différents modules.

L'objectif est de proposer une solution robuste, évolutive et adaptée aux besoins d'un système de point de vente moderne basé sur la lecture de codes-barres.

2. Architecture générale du système

Le système est composé de trois couches principales :

1. Couche Présentation (Frontend)

La couche présentation est développée avec React et Vite. Elle est organisée en composants réutilisables et gère l'ensemble des interactions utilisateur. React Router assure la navigation entre les différentes pages de l'application, tandis que les communications HTTP avec le backend sont réalisées à l'aide des requêtes fetch ou des services JavaScript dédiés. L'état global de l'application (informations utilisateur, token JWT) est géré via Context API, permettant de centraliser les données d'authentification et de les partager entre les différents composants.

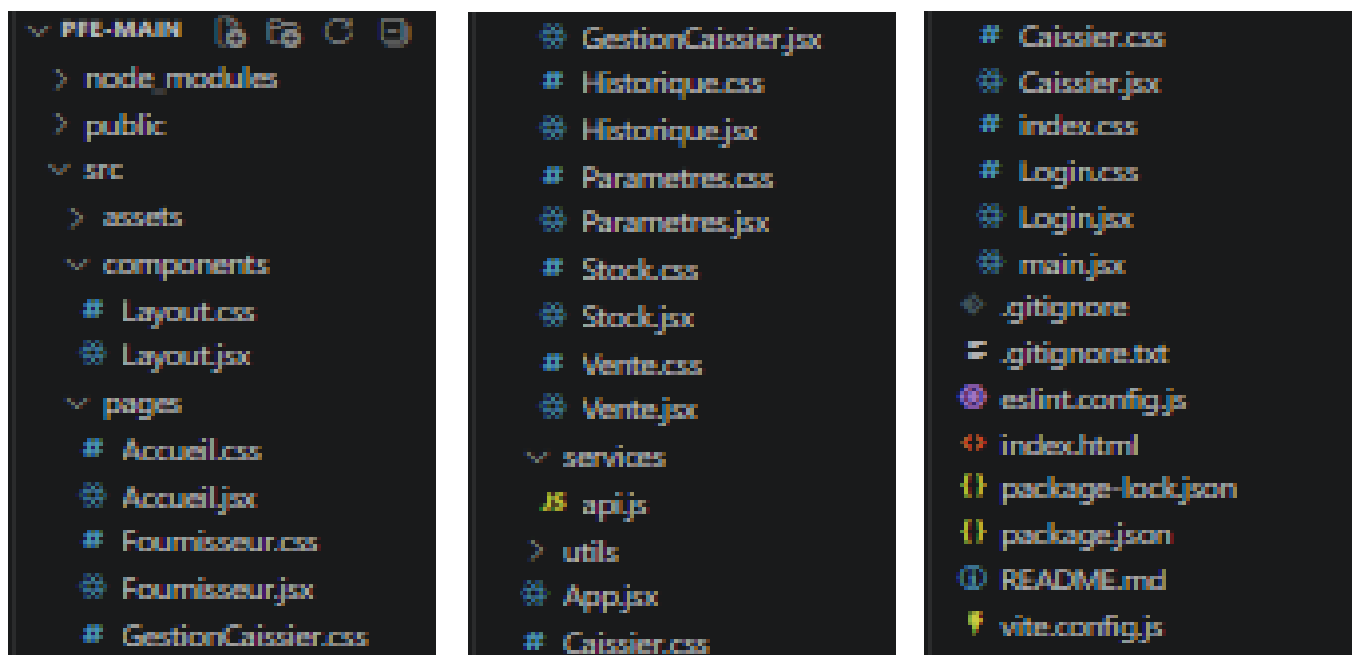


Figure 3. 1 Architecture Frontend React + Vite

2. Couche Métier (Backend)

Le backend de LibrairyPro est développé avec Spring Boot et suit le patron d'architecture **MVC (Modèle-Vue-Contrôleur)** adapté aux API REST. Cette architecture organise le code en couches bien distinctes, garantissant une séparation claire des responsabilités et facilitant la maintenabilité du système.

L'architecture backend est organisée comme suit :

- **Couche Contrôleur (Controller)** : expose les endpoints REST accessibles par le frontend React. Chaque contrôleur correspond à un module fonctionnel du système (ProduitController, VenteController, AuthController, UtilisateurController, etc.). Les contrôleurs reçoivent les requêtes HTTP, délèguent le traitement à la couche Service, et retournent les réponses JSON.
- **Couche Service** : contient la logique métier du système. Les services orchestrent les opérations complexes telles que la validation d'une vente, la mise à jour du stock, ou la génération des statistiques du dashboard. C'est dans cette couche que sont appliquées les règles de gestion.
- **Couche Repository (DAO)** : gère les accès à la base de données MySQL via JPA/Hibernate. Chaque repository étend l'interface JpaRepository de Spring Data, offrant automatiquement les opérations CRUD ainsi que la possibilité de définir des requêtes personnalisées.
- **Couche Modèle (Entités JPA)** : représente les entités du domaine métier mappées vers les tables de la base de données MySQL (Produit, Vente, Utilisateur, Categorie, Fournisseur, etc.).
- **Spring Security + JWT Filter** : intercepte chaque requête HTTP entrante, valide le token JWT présent dans l'en-tête Authorization, et autorise ou rejette l'accès selon le rôle de l'utilisateur (ADMIN ou CAISSIER).

Le flux de traitement d'une requête suit systématiquement le chemin suivant :

Frontend React → Controller REST → Service métier → Repository JPA → MySQL → Réponse JSON → Frontend



Figure 3. 2 Architecture backend Spring Boot

3. Couche Données

La couche données repose sur une base de données relationnelle MySQL. Les données sont organisées en tables liées par des clés étrangères, garantissant l'intégrité référentielle et permettant des requêtes SQL complexes pour les statistiques et rapports.

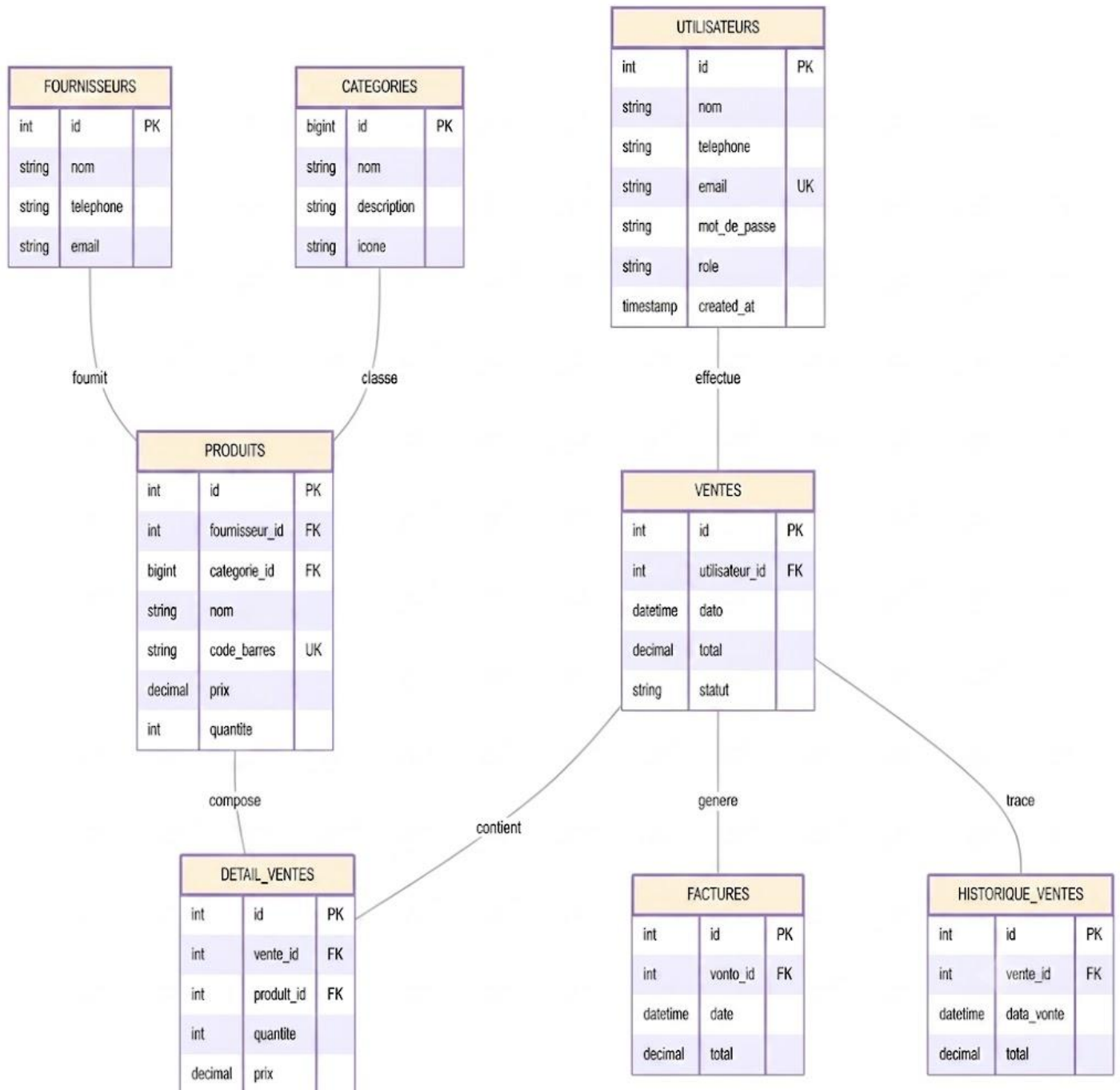


Figure 3. 3 Structure de la base de données

3. Modélisation UML du système

3.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation représente les interactions entre les acteurs du système (Administrateur et Caissier) et les fonctionnalités offertes par l'application LibrairyPro. Il permet d'identifier clairement les droits et responsabilités de chaque profil utilisateur.

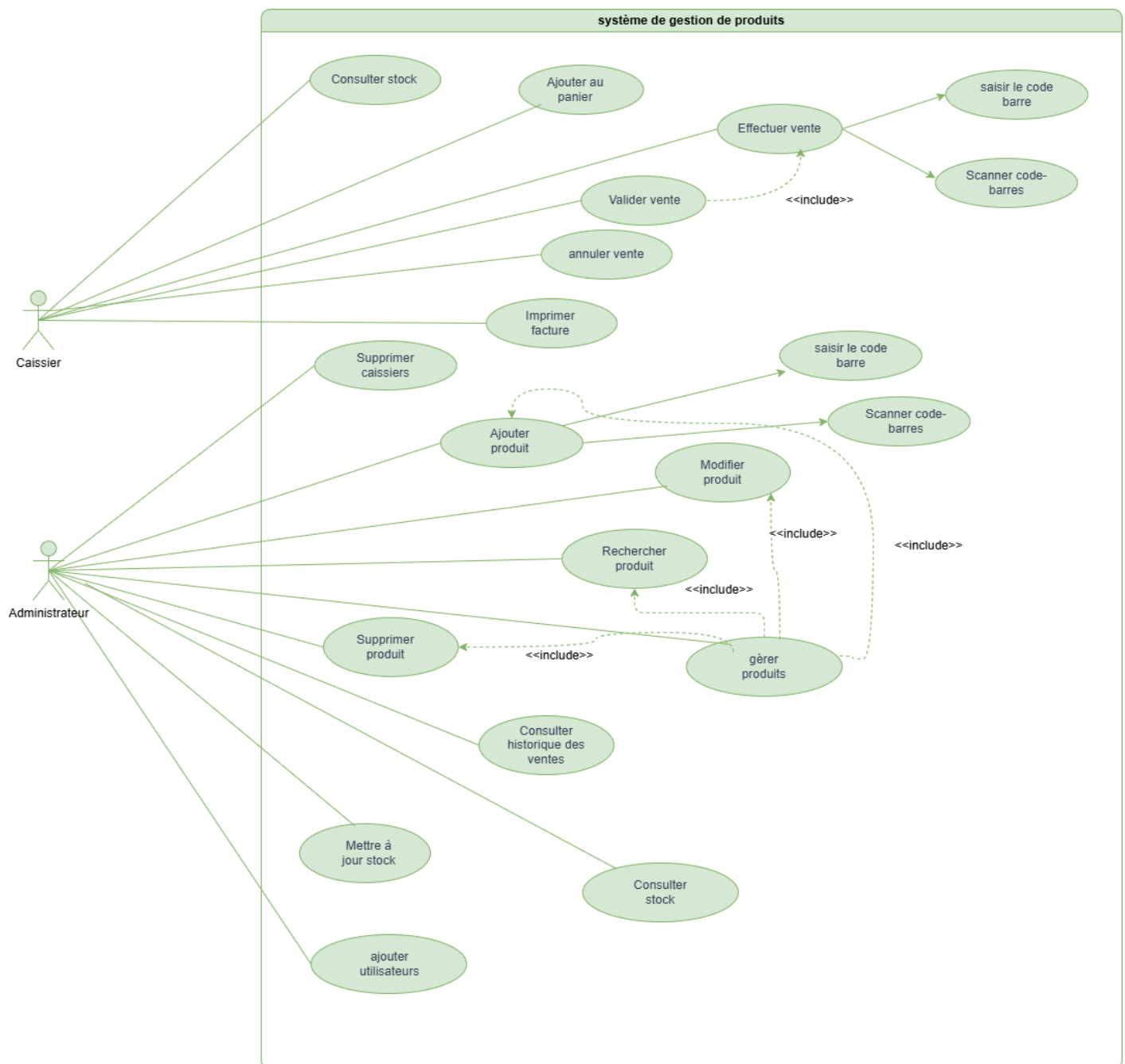


Figure 3. 4 Diagramme de cas d'utilisation

3.2 Diagramme de classes

Le diagramme de classes présente la structure statique du système en décrivant les entités principales, leurs attributs, leurs méthodes ainsi que les relations qui les lient. Il reflète directement la couche Modèle de l'architecture MVC adoptée dans LibrairyPro, et correspond au mapping objet-relationnel implémenté via JPA/Hibernate vers la base de données MySQL.

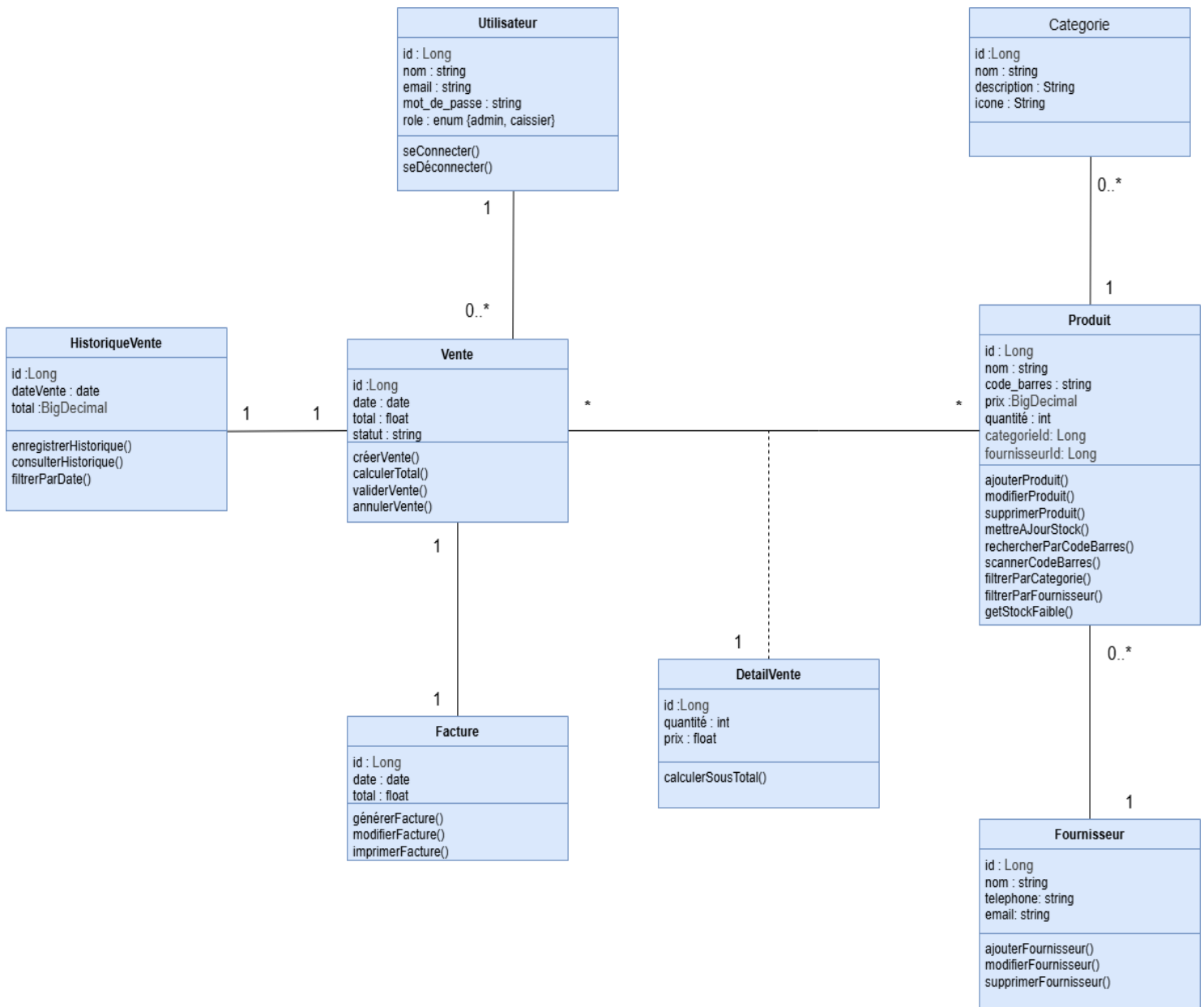


Figure 3. 5 Diagramme de classes

3.3 Diagrammes de séquence

Les diagrammes de séquence illustrent le comportement dynamique du système en représentant les échanges de messages entre les composants au fil du temps. Ils mettent en évidence le rôle de chaque couche de l'architecture MVC : la Vue (composants React), le Contrôleur (REST Controllers Spring Boot), et le Modèle (Services + Repositories JPA + MySQL).

Diagramme de séquence 1 : Authentification utilisateur (Login JWT)

Ce diagramme décrit le flux d'authentification depuis la saisie des identifiants par l'utilisateur jusqu'à la génération et au stockage du token JWT côté frontend.

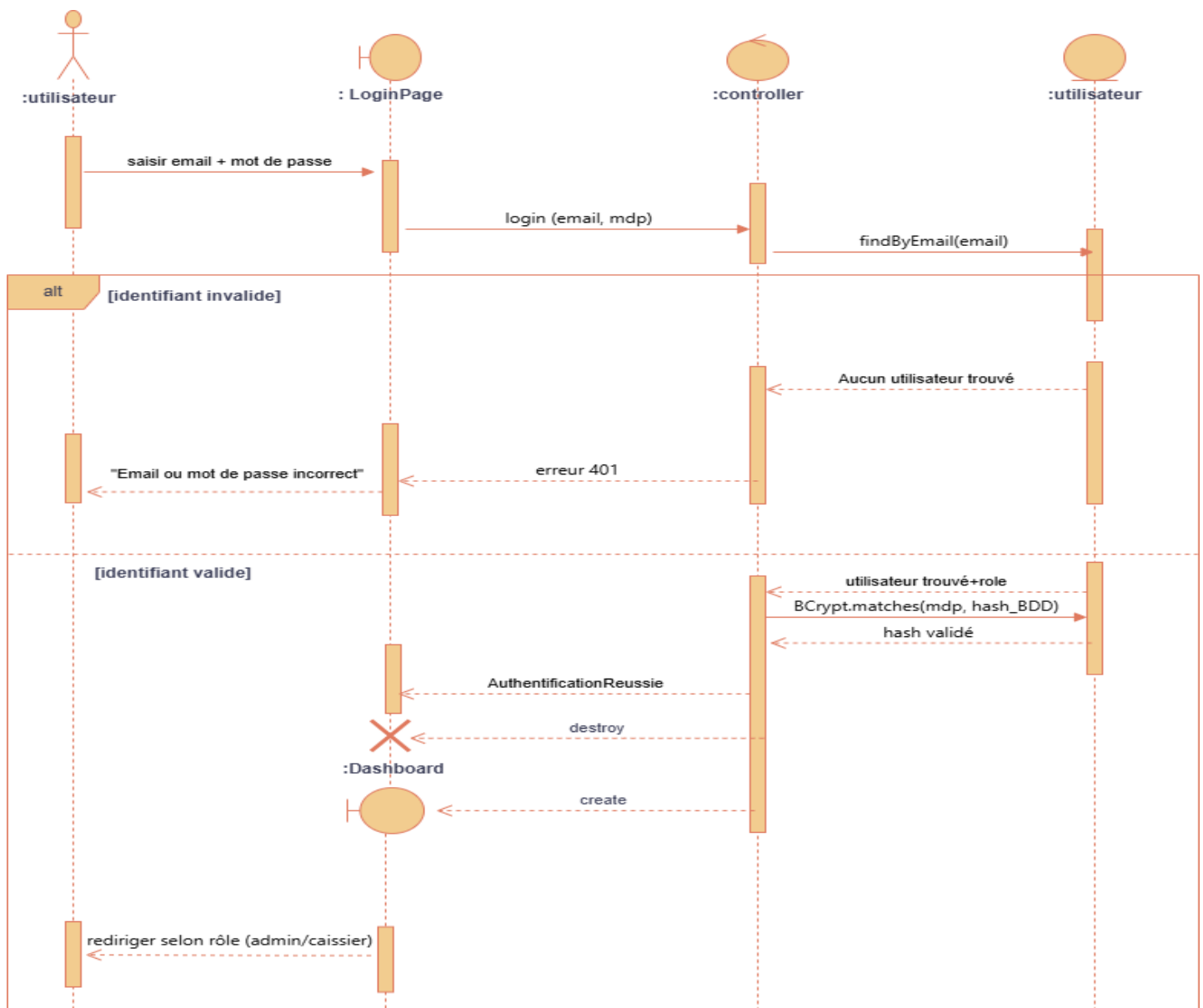


Figure 3. 6 Diagramme de séquence authentification

Diagramme de séquence 2 : Processus de vente avec scan de code-barres

Ce diagramme illustre le flux complet d'une opération de vente : scan du code-barres par le caissier, identification du produit via l'API REST, ajout au panier, validation de la vente, mise à jour du stock et génération de la facture PDF.

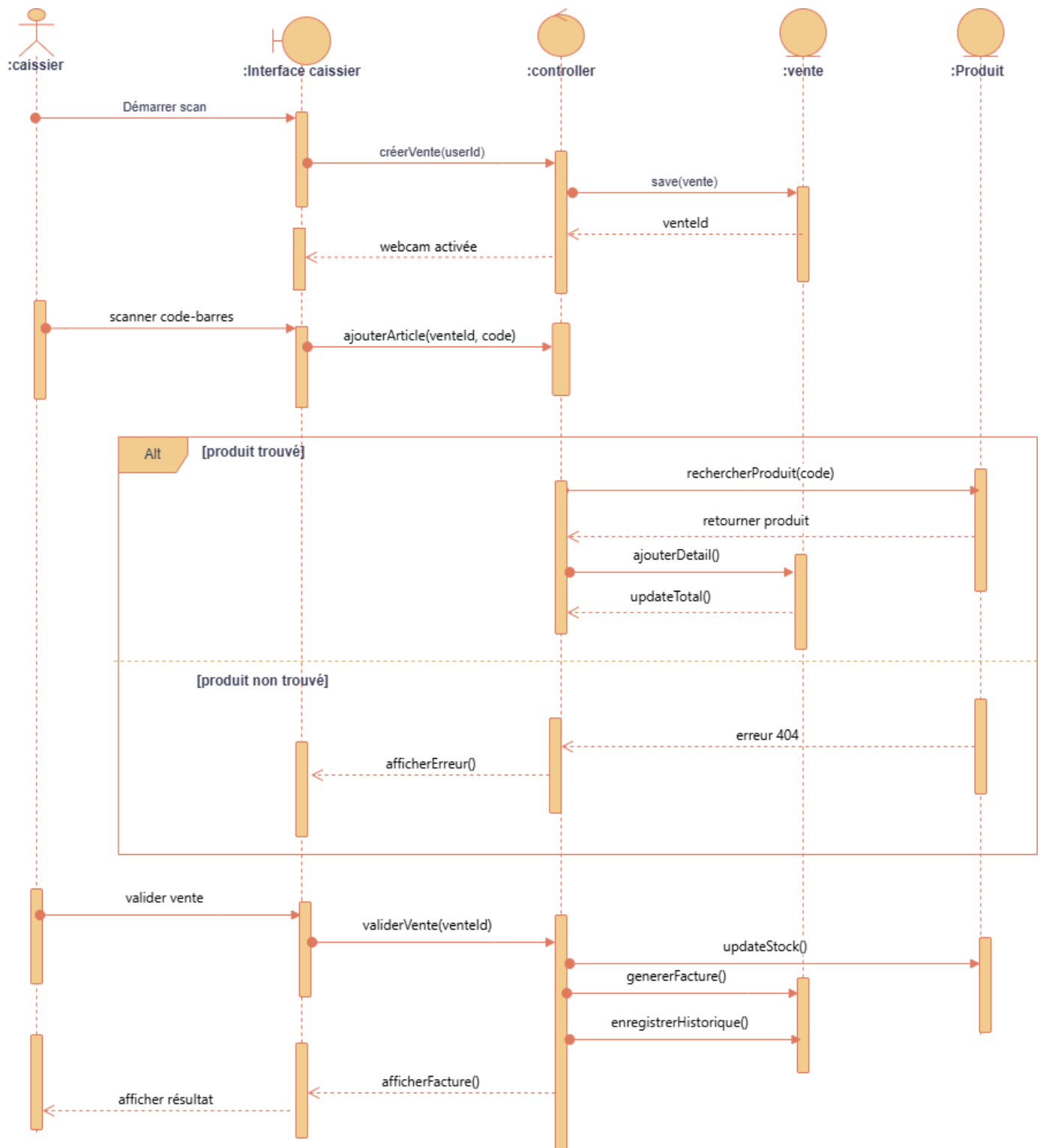


Figure 3. 7 Diagramme de séquence vente avec scan du code barre

Diagramme de séquence 3 : Processus d'ajout de produit avec scan de code-barres

Ce diagramme illustre le flux complet d'une opération d'ajout d'un produit par l'administrateur.

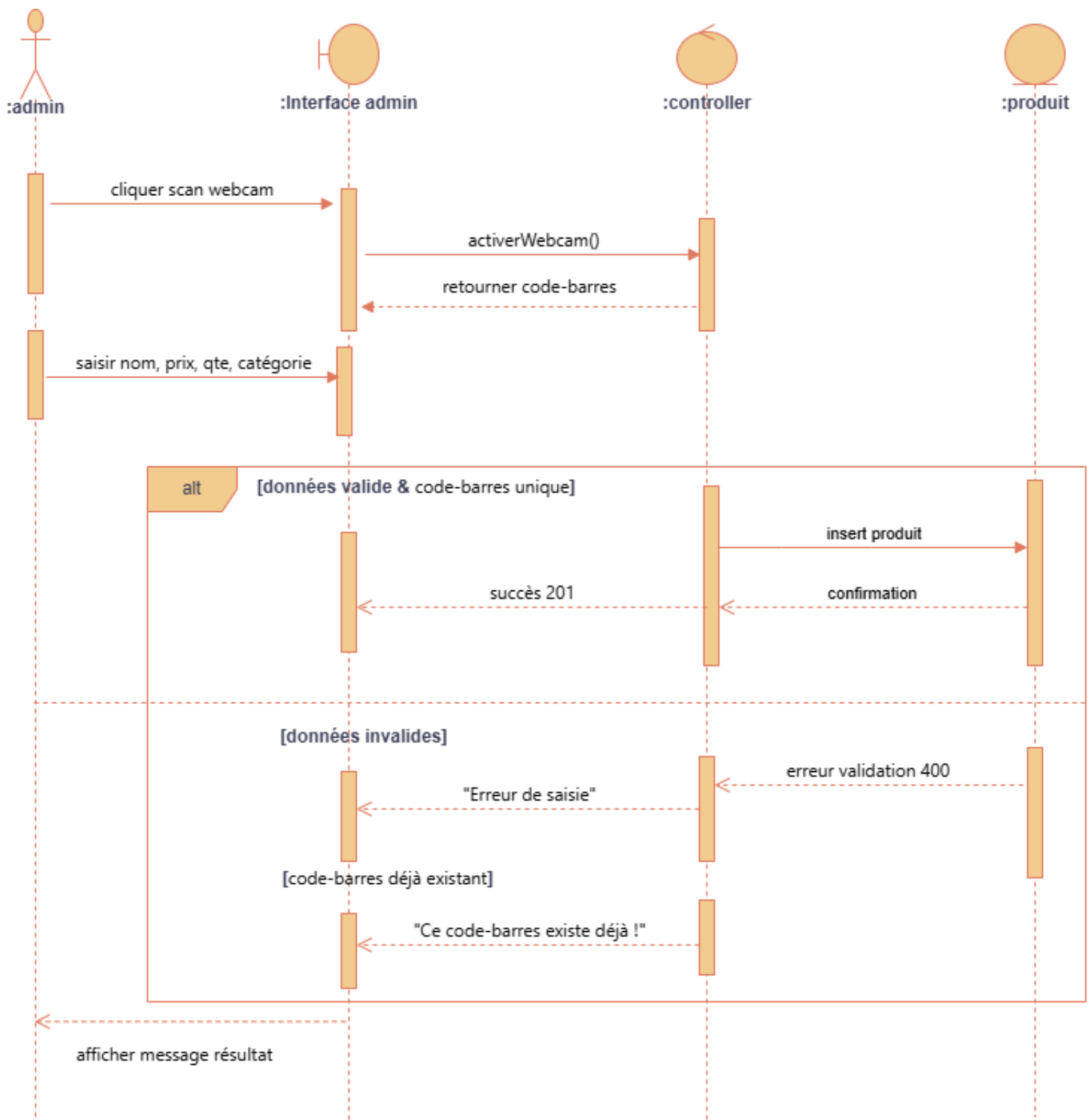


Figure 3. 8 Diagramme de séquence d'ajout de produit avec scan de code-barres

4. Flux de données entre composants

Le fonctionnement du système suit le flux suivant :

- L'utilisateur interagit avec l'interface React + Vite
- Une requête HTTP est envoyée au backend Spring Boot
- Le backend traite la logique métier
- Les données sont stockées dans MySQL
- Une réponse JSON est renvoyée au frontend

5. Système de code-barres

Chaque produit dans la base de données possède un champ `code_barres` UNIQUE (format EAN-13 ou ISBN pour les livres). Lors d'une opération de vente, le caissier saisit ou scanne le code-barres, le système interroge la table produits via une requête REST GET `/api/produits/barcode/{code}`, le produit est identifié et automatiquement ajouté à la facture en cours avec décrémentation du stock en temps réel.

6. Génération de facture PDF

La génération de la facture PDF dans LibrairyPro repose sur la méthode `window.print()` du navigateur, combinée avec une feuille de style CSS d'impression (`@media print`). Cette approche a été choisie pour sa simplicité d'implémentation et sa compatibilité universelle avec tous les navigateurs modernes, sans nécessiter de bibliothèque tierce supplémentaire. Le CSS d'impression masque tous les éléments de navigation et n'affiche que le contenu de la facture avec un tampon 'PAYÉ'. Comme alternative, la bibliothèque jsPDF aurait pu être utilisée pour générer un fichier PDF binaire côté client, offrant plus de contrôle sur la mise en page mais ajoutant une dépendance supplémentaire.

7. Authentification et sécurité (JWT)

Le système d'authentification se déroule en plusieurs étapes :

- L'utilisateur envoie ses identifiants (email + mot de passe) via POST /api/auth/login.
- Spring Security valide les identifiants contre la base de données (mot de passe haché BCrypt).
- En cas de succès, un token JWT est généré et retourné au frontend.
- Le frontend stocke ce token et l'inclut dans l'en-tête Authorization: Bearer {token} de chaque requête API.
- Spring Security vérifie le token à chaque requête et applique les restrictions de rôle (ADMIN ou CAISSIER).

8. Conclusion du chapitre

Ce chapitre a présenté l'architecture globale de LibrairyPro ainsi que les choix techniques adoptés et justifiés par rapport à des alternatives. L'utilisation d'une architecture moderne en trois couches basée sur React, Spring Boot et MySQL permet d'assurer performance, sécurité et évolutivité du système.

CHAPITRE 4 RÉALISATION



1. Introduction du chapitre

Ce chapitre présente la phase de réalisation du projet **LibrairyPro**, incluant les outils utilisés, l'implémentation technique des différentes fonctionnalités ainsi que les interfaces développées pour les deux profils utilisateurs (administrateur et caissier).

L'objectif est de montrer comment les choix conceptuels présentés précédemment ont été concrètement implémentés.

2. Outils et technologies utilisés

- Technologies Frontend

L'interface utilisateur a été développée avec React 19 et configurée à l'aide de Vite afin de bénéficier d'un environnement de développement moderne et performant. L'application est organisée en composants réutilisables et communique avec le backend via des appels HTTP. La navigation entre les différentes pages est assurée par React Router, permettant la mise en place de routes adaptées aux fonctionnalités de l'application. Le projet intègre également la bibliothèque html5-qrcode pour la lecture de code-barres ainsi que jsPDF et jsPDF-AutoTable pour la génération de documents PDF, notamment les factures. La mise en page repose sur les technologies CSS modernes telles que Flexbox et Grid afin d'assurer une interface claire et cohérente.



Figure 4. 1 Technologies utilisées Frontend

- Technologies Backend

La sécurité de l'application a été mise en œuvre à l'aide de Spring Security et d'un mécanisme d'authentification basé sur JWT (JSON Web Token). Un filtre personnalisé (JwtFilter), hérité de OncePerRequestFilter, intercepte chaque requête HTTP afin de vérifier la présence et la validité du token transmis dans l'en-tête Authorization. Après validation, les informations de l'utilisateur (email et rôles) sont extraites du token et enregistrées dans le contexte de sécurité de Spring Security. Les droits d'accès aux différentes ressources sont ensuite contrôlés selon les rôles attribués aux utilisateurs, notamment ADMIN et CAISSIER. L'application adopte une architecture stateless, où aucune session n'est conservée côté serveur.



Figure 4. 2 Technologies utilisées Backend

- Bases de données

La base de données du projet a été implémentée sous MySQL et organisée autour de plusieurs tables relationnelles représentant les différentes entités métier de l'application, notamment les utilisateurs, produits, catégories, fournisseurs, ventes, détails de ventes, factures et historiques de ventes. Les relations entre ces tables sont assurées par des clés étrangères garantissant l'intégrité référentielle des données. Le script de création de la base de données, comprenant la définition des tables, des contraintes et des données initiales, est regroupé dans le fichier BDD.sql du projet. Du côté du backend, chaque table est associée à une entité Java grâce à JPA/Hibernate et aux annotations telles que @Entity, @Table et @Column, assurant la correspondance entre le modèle objet et le schéma relationnel.

Description de la base de données MySQL

Table	Colonnes principales	Rôle	Relations
utilisateurs	id, nom, email, mot_de_passe (BCrypt), role (ENUM admin/caissier), created_at	Gestion des comptes utilisateurs avec authentification sécurisée.	Référencée par : ventes (utilisateur_id)
produits	id, nom, code_barres (UNIQUE), prix, quantite, fournisseur_id, categorie_id	Catalogue complet des produits avec code-barres unique.	FK vers : fournisseurs, categories
categories	id, nom, description, icone	Classification des produits en 8 catégories (Livres, Fournitures, etc.).	Référencée par : produits
fournisseurs	id, nom, telephone, email	Informations des fournisseurs de produits.	Référencée par : produits
ventes	id, utilisateur_id, date, total, statut (ENUM en_cours/validee/annulee)	Enregistrement des transactions de vente.	FK vers : utilisateurs. Référencée par : detail_ventes, factures
detail_ventes	id, vente_id, produit_id, quantite, prix (au moment de la vente)	Lignes de détail d'une vente (articles vendus).	FK vers : ventes, produits
factures	id, vente_id (UNIQUE), date, total	Relation 1-1 avec une vente validée.	FK vers : ventes
historique_ventes	id, vente_id, date_vente, total	Archive des ventes pour consultation historique.	FK vers : ventes

Table 3 Description de la base de données MySQL

- Outils de développement

L'environnement technique retenu pour la réalisation de ce projet s'appuie sur une sélection d'outils de développement modernes et complémentaires, garantissant une efficacité optimale à chaque étape du cycle de vie de l'application. Le développement de la partie **Frontend** est assuré sous **VS Code**, un éditeur de code léger, hautement extensible pour l'intégration d'interfaces utilisateur. Côté **Backend**, le choix s'est porté sur **IntelliJ IDEA**, un environnement de développement intégré (IDE) robuste, particulièrement performant pour la gestion de l'architecture serveur et des langages orientés objet. La modélisation, la gestion et la requête de la base de données relationnelle sont centralisés sur **MySQL Workbench**. Pour valider la conformité de la logique métier, **Postman** est utilisé comme outil de référence afin de tester rigoureusement l'ensemble des points de terminaison (endpoints) de l'API. Enfin, l'intégrité du code source, le suivi des modifications et la collaboration sont orchestrés de manière fluide grâce au système de contrôle de version **Git**, couplé à la plateforme d'hébergement **GitHub**.



Figure 4. 3 Outils de développement

3. Présentation des interfaces de l'application

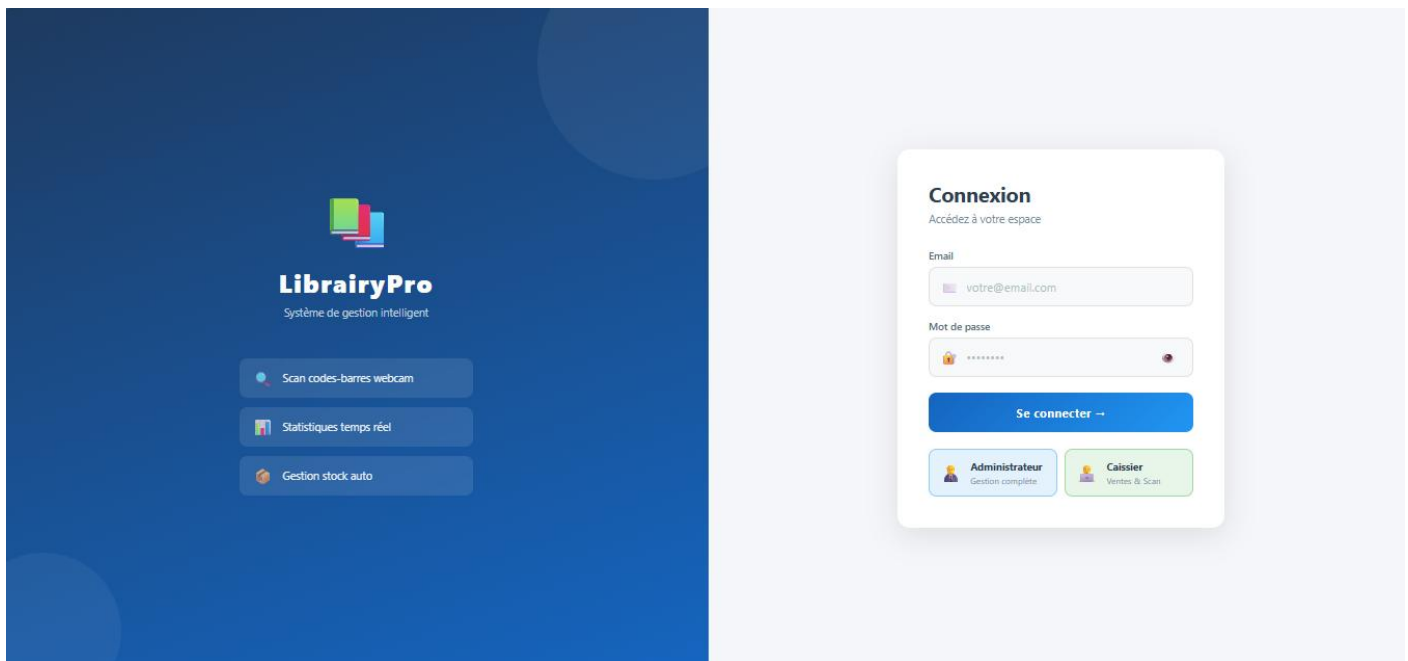


Figure 4. 4 Page initiale

3.1 Système de caisse (POS)

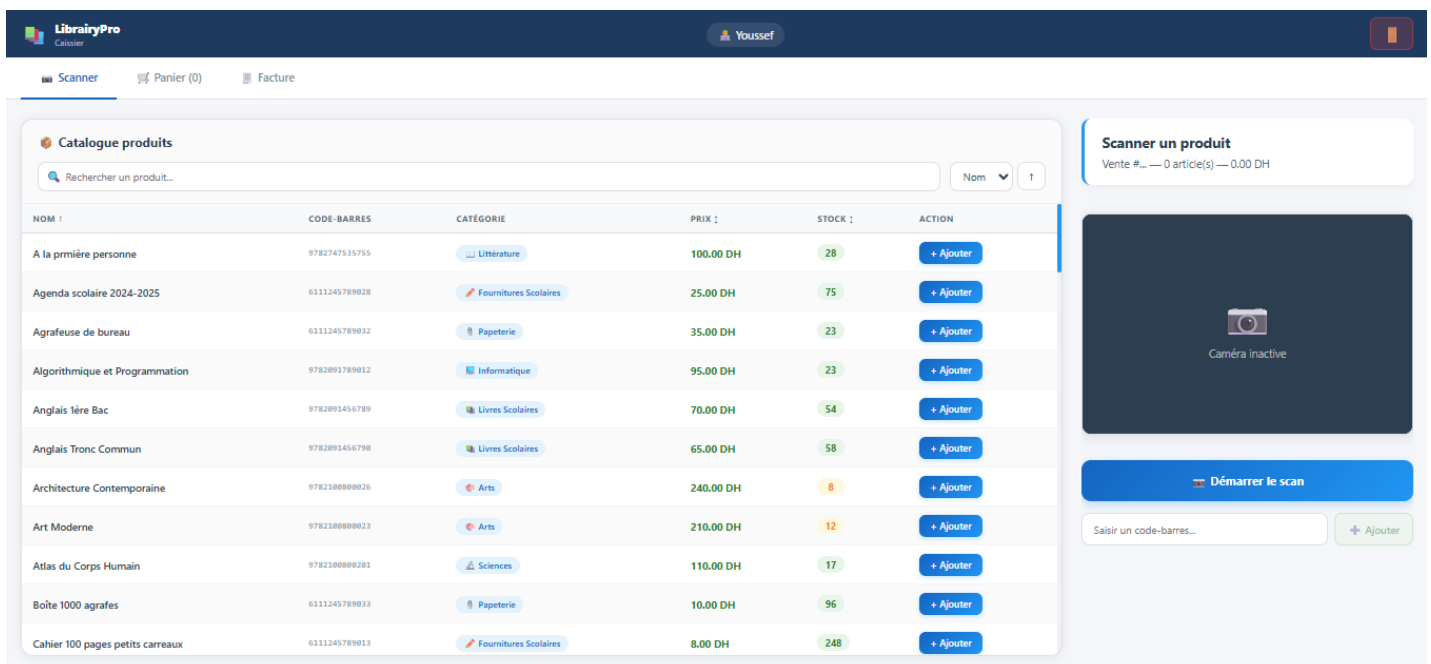


Figure 4. 5 Page d'accueil du caissier permettant la gestion des produits et la lecture des code-barres [scan code-barre off]

LibrairyPro

Youssef

Scanner

Panier (3)

Facture

Catalogue produits

Rechercher un produit...

Nom 1

NOM :	CODE-BARRES	CATÉGORIE	PRIX :	STOCK :	ACTION
A la première personne	9782747535755	Littérature	100.00 DH	27	+ Ajouter
Agenda scolaire 2024-2025	6111245789028	Fournitures Scolaires	25.00 DH	74	+ Ajouter
Agrafeuse de bureau	6111245789032	Papeterie	35.00 DH	23	+ Ajouter
Algorithmique et Programmation	9782091789012	Informatique	95.00 DH	22	+ Ajouter
Anglais 1ère Bac	9782091456789	Livres Scolaires	70.00 DH	54	+ Ajouter
Anglais Tronc Commun	9782091456790	Livres Scolaires	65.00 DH	58	+ Ajouter
Architecture Contemporaine	9782100000026	Arts	240.00 DH	8	+ Ajouter
Art Moderne	9782100000023	Arts	210.00 DH	12	+ Ajouter
Atlas du Corps Humain	97821000000201	Sciences	110.00 DH	17	+ Ajouter
Boîte 100 agrafes	6111245789033	Papeterie	10.00 DH	96	+ Ajouter
Cahier 100 pages petits carreaux	6111245789013	Fournitures Scolaires	8.00 DH	248	+ Ajouter

Scanner un produit

Vente #47 — 3 article(s) — 220.00 DH

Arrêter le scan

Saisir un code-barres... + Ajouter

Voir le panier --

Figure 4. 6 Page d'accueil du caissier permettant la gestion des produits et la lecture des code-barres [scan code-barre on]

LibrairyPro

Youssef

Scanner

Panier (3)

Facture

Récapitulatif — Vente #47

A la première personne

9782747535755

100 DH

Agenda scolaire 2024-2025

6111245789028

25 DH

Algorithmique et Programmation

9782091789012

95 DH

Articles :

3

TOTAL :

220.00 DH

Scanner plus

Annuler

Valider la vente

Figure 4. 7 Page de panier

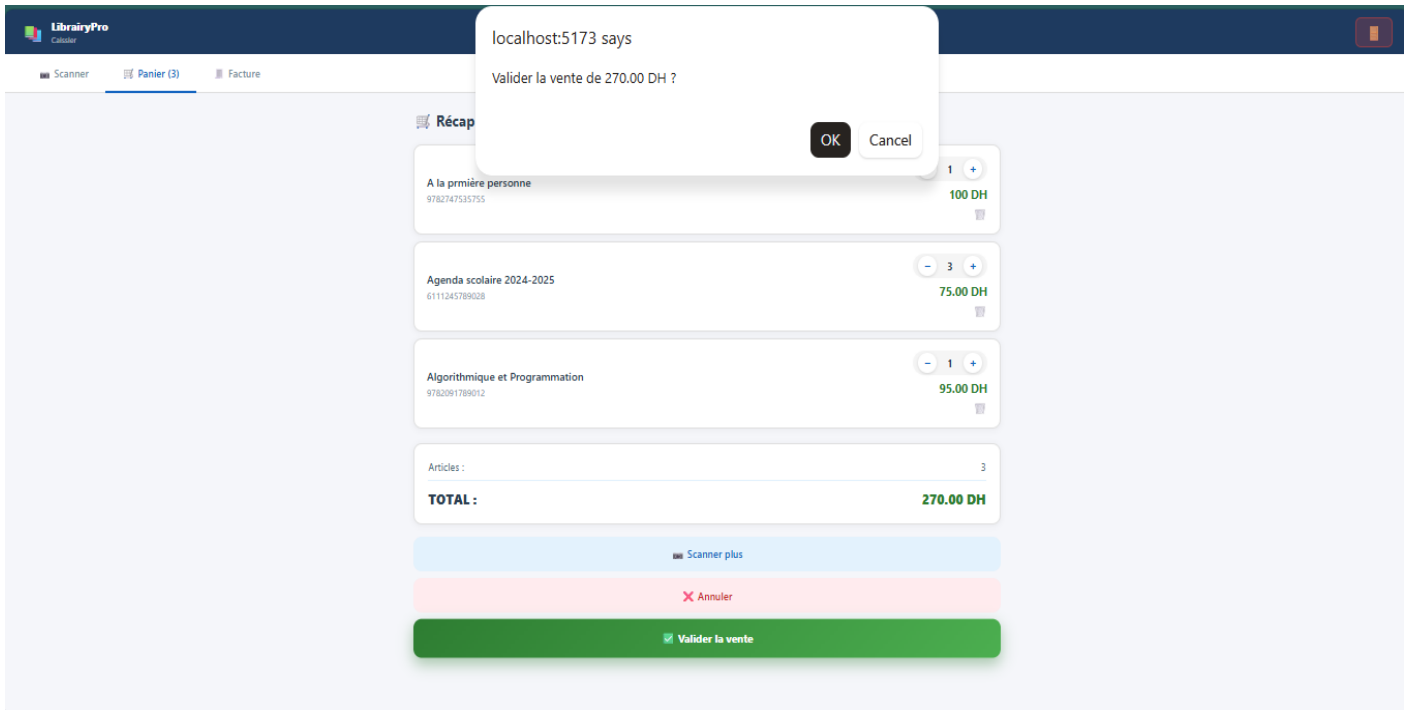


Figure 4. 8 Alerte de validation de panier/vente

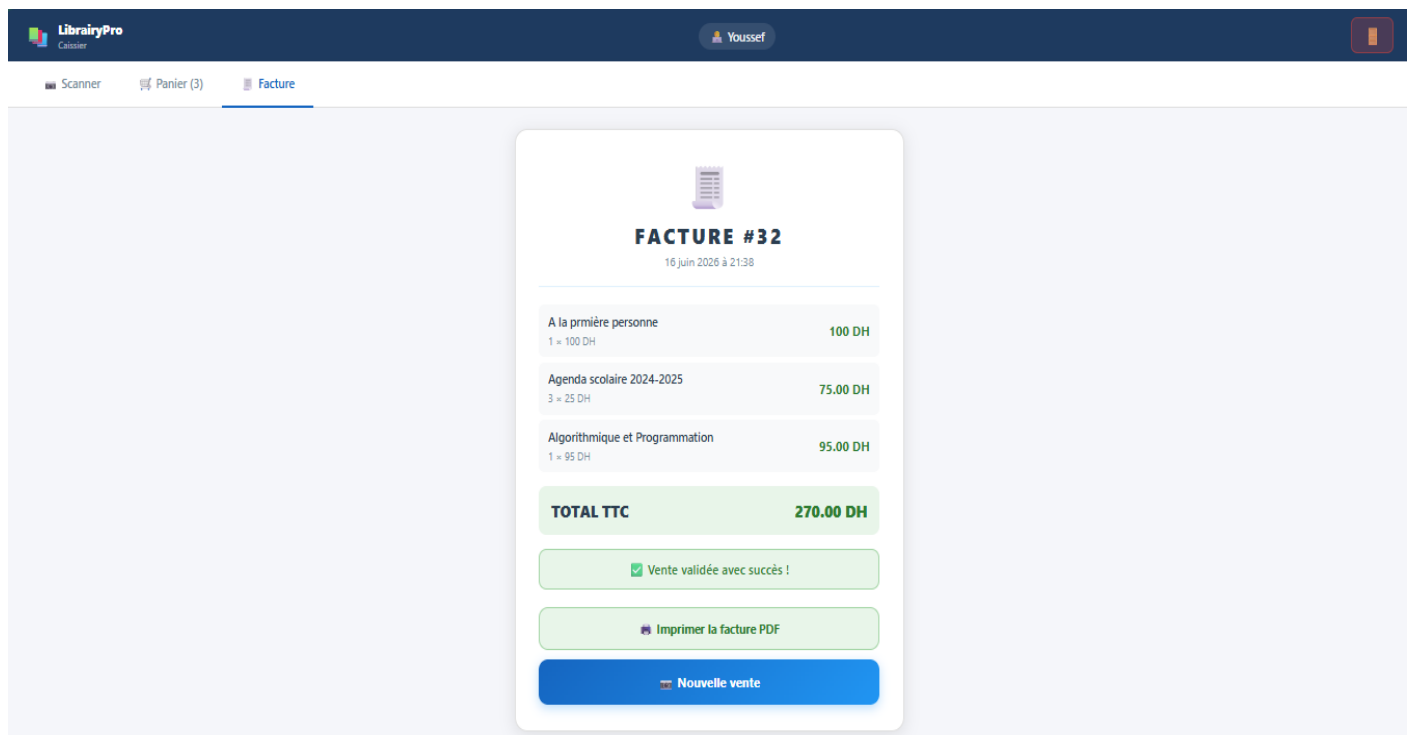


Figure 4. 9 Page du facture

FACTURE

N° Facture : 32

Date : 16/06/2026

Informations

Vente N° : 47

Date et heure : 16/06/2026 21:41:07

Produit	Code-barres	Qté	Prix	Sous-total
A la première personne	9782747535755	1	100.00 DH	100.00 DH
Agenda scolaire 2024-2025	6111245789028	3	25.00 DH	75.00 DH
Algorithmique et Programmation	9782091789012	1	95.00 DH	95.00 DH

TOTAL : 270.00 DH

Merci pour votre achat.

3.2 Dashboard administrateur

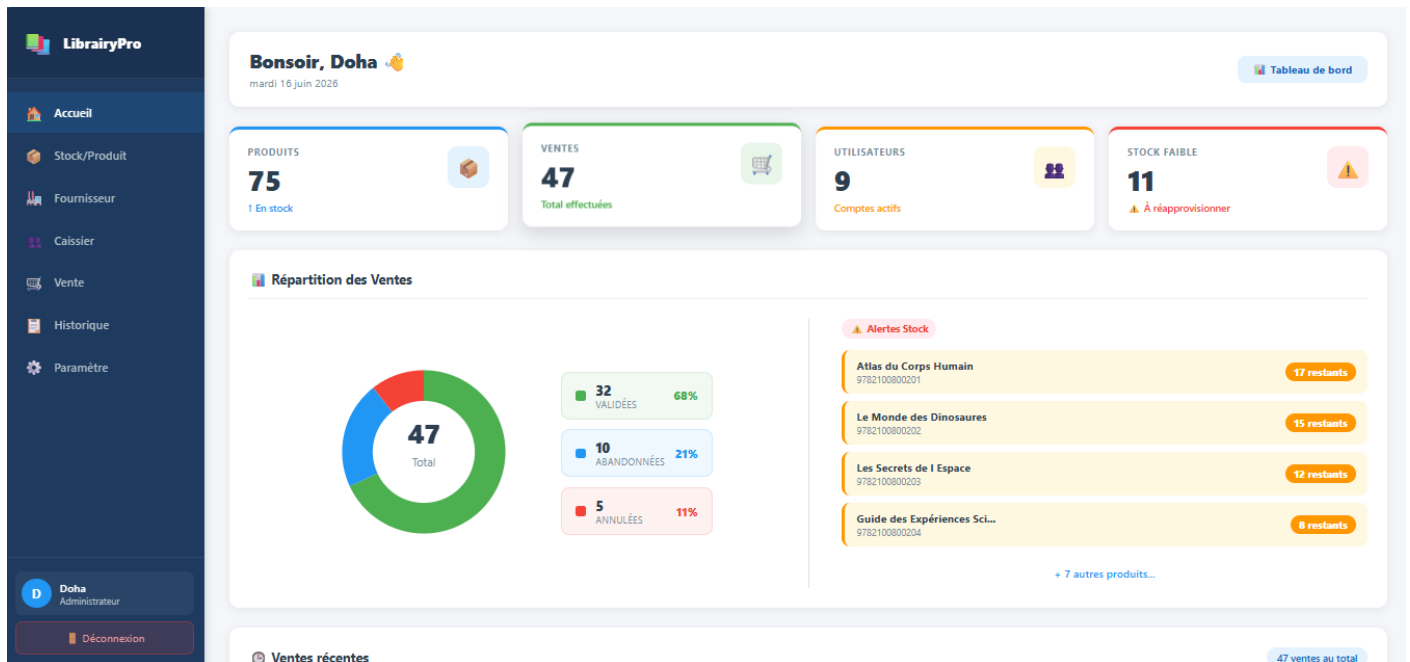


Figure 4. 11 Page d'accueil du dashboard admin 1

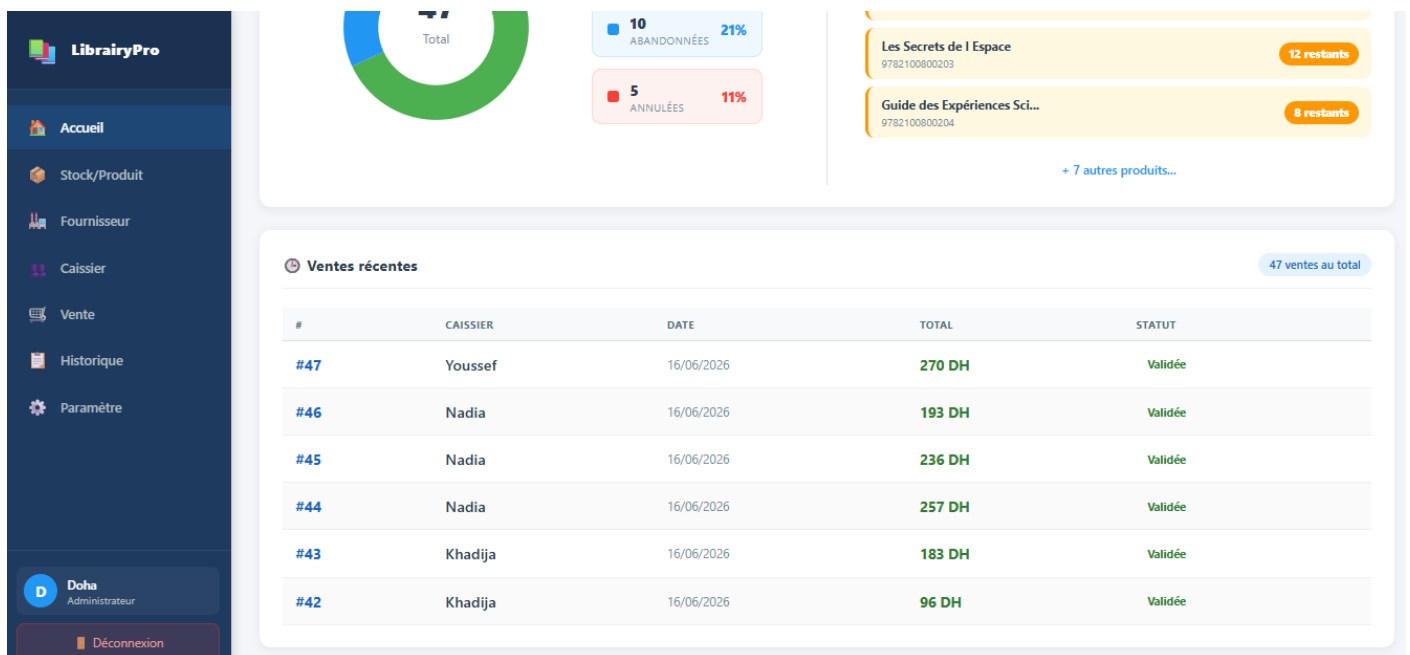


Figure 4. 12 Page d'accueil du dashboard admin 2

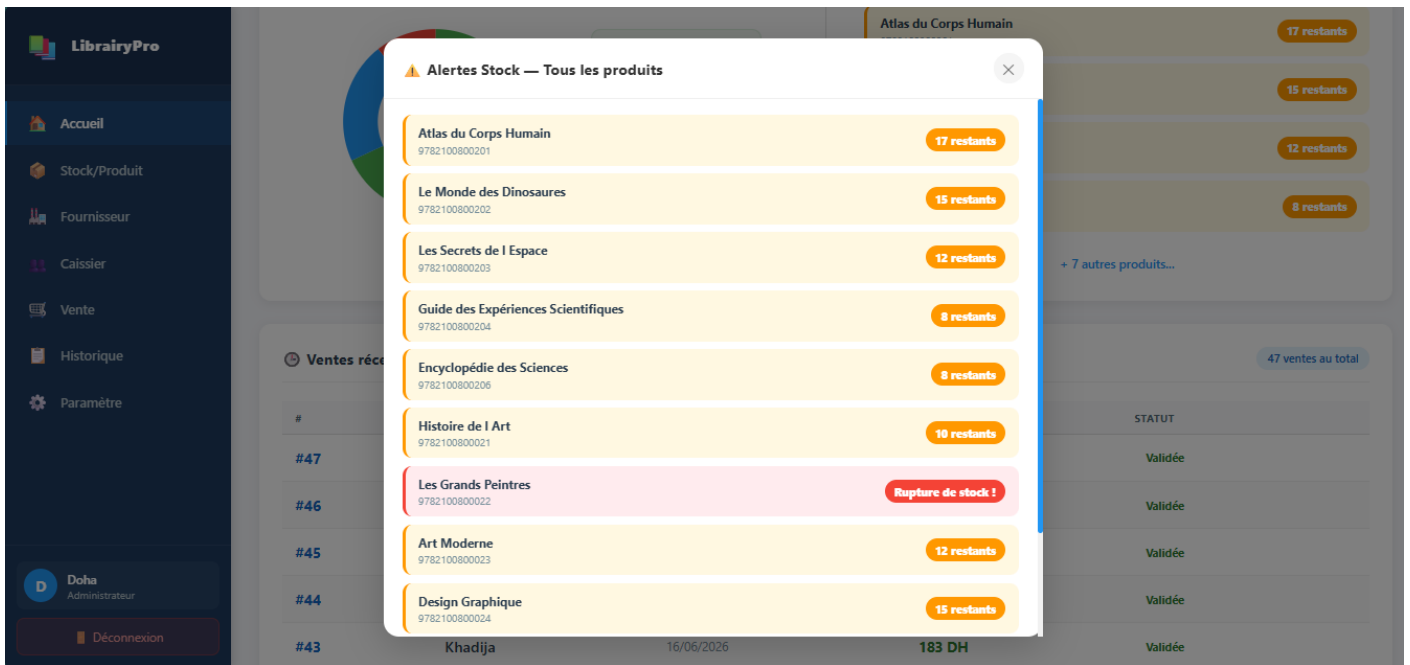


Figure 4. 13 Page d'accueil du dashboard admin 3 [Alerte du stock]

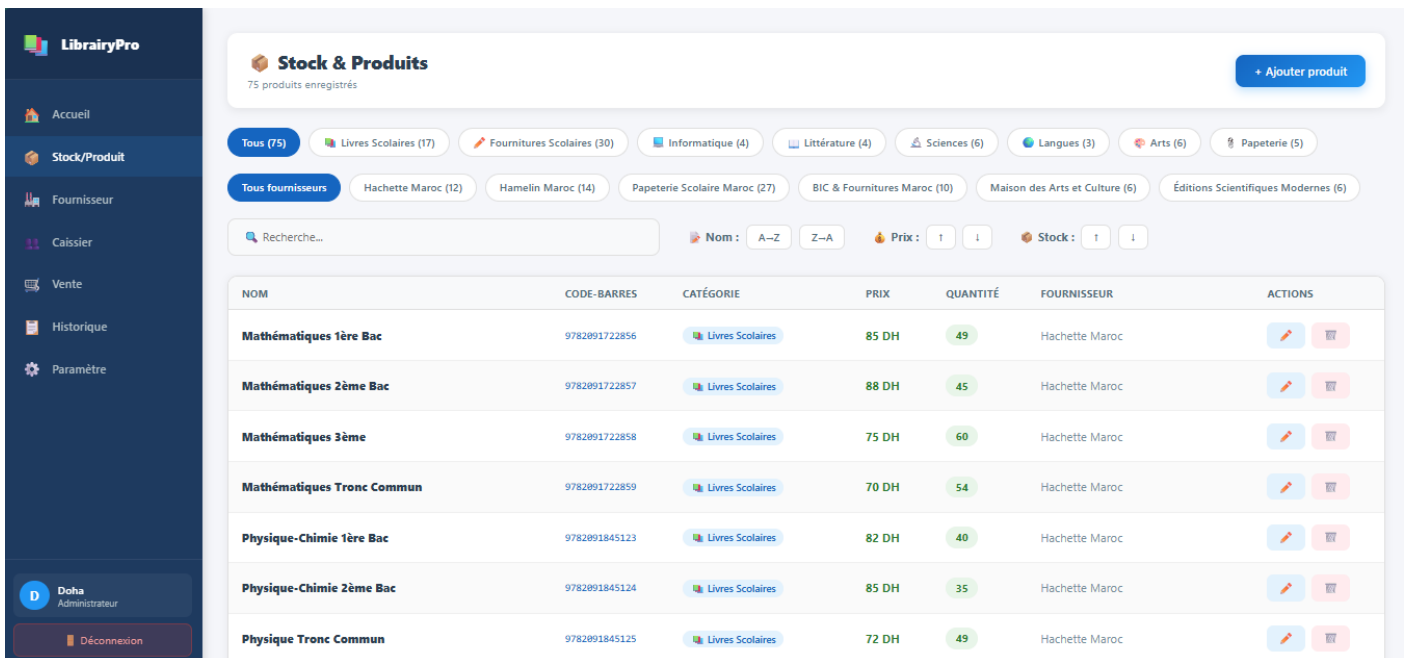


Figure 4. 14 Page de stock/produit du dashboard admin 1

Figure 4. 15 Page de stock/produit [ajout du produit] du dashboard admin 2

Figure 4. 16 Page de fournisseur du dashboard admin 1

LibrairyPro

Accueil

Stock/Produit

Fournisseur

Caissier

Vente

Historique

Paramètre

D Doha Administrateur

Déconnexion

Fournisseurs

6 fournisseurs enregistrés

Fermer

Nouveau fournisseur

Nom *

Ex: Hachette Maroc

Téléphone

Ex: 0612345678 ou +212612345678

Email

Ex: contact@exemple.com

Ajouter

Annuler

H Hachette Maroc

0612345678

contact@hachette.ma

Modifier

Supprimer

H Hamelin Maroc

0623456789

hamelin@papeterie.ma

Modifier

Supprimer

P Papeterie Scolaire Maroc

0634567890

contact@papeterie.ma

Modifier

Supprimer

B BIC & Fournitures Maroc

0645678901

contact@bicmaroc.ma

Modifier

Supprimer

M Maison des Arts et Culture

E Éditions Scientifiques Mode...

Figure 4. 17 Page de fournisseur [ajout du fournisseur] du dashboard admin 2

LibrairyPro

Accueil

Stock/Produit

Fournisseur

Caissier

Vente

Historique

Paramètre

D Doha Administrateur

Déconnexion

Gestion des Caissiers

8 caissier(s)

Ajouter

UTILISATEUR	TÉLÉPHONE	EMAIL	VENTES	ACTION
F Fatima Zahra	0671523498	fatima@librairie.ma	2 vente(s)	Modifier Supprimer
Y Youssef	0682347156	youssef@librairie.ma	6 vente(s)	Modifier Supprimer
K Khadija	0693765214	khadija@librairie.ma	4 vente(s)	Modifier Supprimer
A Anas	0664218973	anas@librairie.ma	2 vente(s)	Modifier Supprimer
K Khalid	0612378945	khalid@librairie.ma	5 vente(s)	Modifier Supprimer
I Imane	0634591267	imane@librairie.ma	6 vente(s)	Modifier Supprimer
H Hamza	0645612378	hamza@librairie.ma	2 vente(s)	Modifier Supprimer
N Nadia	0656723489	nadia@librairie.ma	5 vente(s)	Modifier Supprimer

Figure 4. 18 Page du caissier du dashboard admin 1

LibrairyPro

Accueil

Stock/Produit

Fournisseur

Caissier

Vente

Historique

Paramètre

D Doha Administrateur

Déconnexion

Gestion des Caissiers

8 caissier(s)

Fermer

Nouveau caissier

Nom complet *

Nom complet

Email *

email@librairie.ma

Téléphone *

Numéro de téléphone

Mot de passe *

Créer le compte

Annuler

UTILISATEUR	TÉLÉPHONE	EMAIL	VENTES	ACTION
F Fatima Zahra	0671523498	fatima@librairie.ma	2 vente(s)	Modifier Supprimer
Y Youssef	0682347156	youssef@librairie.ma	6 vente(s)	Modifier Supprimer
K Khadija	0693765214	khadija@librairie.ma	4 vente(s)	Modifier Supprimer
A Anas	0664218973	anas@librairie.ma	2 vente(s)	Modifier Supprimer

Figure 4. 19 Page du caissier [ajout du caissier] du dashboard admin 2

LibrairyPro

Accueil

Stock/Produit

Fournisseur

Caissier

Vente

Historique

Paramètre

D Doha Administrateur

Déconnexion

Gestion des Ventes

47 ventes — Total validées : 6102.50 DH

32 Validées

10 Abandonnées

5 Annulées

Tous

Abandonnées

Validées

Annulées

y

#	CAISSIER	DATE	TOTAL	STATUT	
#47	Youssef	16/06/2026	270 DH	Validée	Détails --
#33	Youssef	16/06/2026	470 DH	Validée	Détails --
#32	Youssef	16/06/2026	0 DH	Abandonnée	Détails --
#31	Youssef	16/06/2026	0 DH	Abandonnée	Détails --
#30	Youssef	16/06/2026	0 DH	Abandonnée	Détails --
#29	Youssef	16/06/2026	360 DH	Annulée	Détails --
#28	Youssef	15/06/2026	275 DH	Abandonnée	Détails --
#6	Youssef	13/06/2026	105 DH	Annulée	Détails --

Vente #47

Caissier

Youssef

Date

16/06/2026

Statut

validée

ARTICLES

A la première personne

100 DH

Agenda scolaire 2024-2025

75 DH

Algorithmique et Programmation

95 DH

TOTAL

270 DH

Figure 4. 20 Page de la vente [avec test du recherche et vue de détails] du dashboard admin

FS BEN M'SICK

59

LibraryPro

Accueil

Stock/Produit

Fournisseur

Caissier

Vente

Historique

Paramètre

Doha

Administrateur

Déconnexion

Historique des Ventes

32 entrées — Total : 6102.50 DH

6102.50 DH

Rechercher par vente ou caissier...

#	VENTE	CAISSIER	DATE/HEURE	TOTAL	STATUT
1	#1	Youssef	13/06/2026 17:17:27	100.5 DH	validee
2	#2	Youssef	13/06/2026 17:17:57	108 DH	validee
3	#3	Youssef	13/06/2026 17:18:37	67.5 DH	validee
4	#4	Youssef	13/06/2026 17:19:17	437.5 DH	validee
5	#7	Khadija	13/06/2026 17:20:53	45 DH	validee
6	#9	Anas	13/06/2026 17:21:47	21.5 DH	validee
7	#10	Anas	13/06/2026 17:22:01	195 DH	validee
8	#11	Khalid	13/06/2026 17:26:15	243 DH	validee
9	#14	Fatima Zahra	13/06/2026 19:45:16	89 DH	validee
10	#15	Fatima Zahra	13/06/2026 19:45:29	258 DH	validee
11	#16	Imane	13/06/2026 19:49:17	38 DH	validee

Figure 4. 21 Page d'historique des ventes du dashboard admin

Paramètres													
Configuration du système													
Profil Administrateur D Doha Administrateur système Email admin@librairie.ma Téléphone 0685943516	Informations Système <table> <tr> <td>Application</td> <td>LibrairyPro v1.0</td> </tr> <tr> <td>Backend</td> <td>Spring Boot 4.0</td> </tr> <tr> <td>Base de données</td> <td>MySQL 8.0</td> </tr> <tr> <td>Frontend</td> <td>React Vite</td> </tr> <tr> <td>Authentification</td> <td>JWT</td> </tr> <tr> <td>Scan</td> <td>html5-qrcode</td> </tr> </table>	Application	LibrairyPro v1.0	Backend	Spring Boot 4.0	Base de données	MySQL 8.0	Frontend	React Vite	Authentification	JWT	Scan	html5-qrcode
Application	LibrairyPro v1.0												
Backend	Spring Boot 4.0												
Base de données	MySQL 8.0												
Frontend	React Vite												
Authentification	JWT												
Scan	html5-qrcode												
À propos de LibrairyPro Système de gestion de librairie : gestion du stock, caisse avec scan webcam, facturation PDF, historique des ventes, gestion multi-utilisateurs. © LibrairyPro 2026	Déconnexion Voulez-vous vous déconnecter de l'application ? Votre session sera terminée. Se déconnecter												

Figure 4. 22 Page du paramètre du dashboard admin

4. Extraits de code

4.1 Entité JPA — Produit (Backend Spring Boot)

L'extrait suivant montre la définition de l'entité Produit mappée vers la table MySQL produits :

```
@Data
@Entity
@Table(name = "produits")
public class Produit {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "fournisseur_id")
    private Fournisseur fournisseur;

    @Column(nullable = false)
    private String nom;

    @Column(name = "code_barres", nullable = false, unique = true)
    private String codeBarres;

    @Column(nullable = false)
    private BigDecimal prix;

    @Column(nullable = false)
    private Integer quantite = 0;

    @ManyToOne
    @JoinColumn(name = "categorie_id")
    private Categorie categorie;
}
```

4.2 Endpoint REST — Recherche par code-barres (Backend Spring Boot)

L'extrait suivant montre l'endpoint REST permettant la recherche d'un produit par son code-barres :

```
@GetMapping("/scan/{codeBarres}")

public ResponseEntity<ProduitResponse> scanner(

    @PathVariable String codeBarres) {

    return ResponseEntity.ok(

        convertirEnResponse(

            produitService.getParCodeBarres (codeBarres)

        ) );

}
```

4.3 Configuration JWT — Spring Security (Backend)

```
@Component
@RequiredArgsConstructor
public class JwtFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsServiceImpl userDetailsService;

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7);

        if (!jwtService.estValide(token)) {
            filterChain.doFilter(request, response);
            return;
        }
    }
}
```

```
String email = jwtService.extraireEmail(token);

UserDetails userDetails = userDetailsService
    .loadUserByUsername(email);

UsernamePasswordAuthenticationToken authentication =
    new UsernamePasswordAuthenticationToken(
        userDetails,
        null,
        userDetails.getAuthorities()
    );

SecurityContextHolder.getContext()
    .setAuthentication(authentication);

filterChain.doFilter(request, response);
}
}
```

4.4 Composant React — Scan de code-barres (Frontend)

```
const startScan = async () => {

    setLoading(true);

    let currentVenteId = venteId;

    if (!currentVenteId) {

        try {

            const v = await creerVente(userId);

            currentVenteId = v.id;

            setVenteId(v.id);

        } catch {

            flash("✖ Erreur création vente !");

            setLoading(false);

            return;

        }

    }

}
```

```

setLoading(false);

setScanning(true);

setTimeout(async () => {

  try {

    const qr = new Html5Qrcode("caissier-qr");

    scannerRef.current = qr;

    await qr.start(

      { facingMode: "environment" },

      { fps: 10, qrbox: { width: 300, height: 150 } },

      async (code) => {

        const now = Date.now();

        if (code === lastScannedRef.current && now - lastTimeRef.current < 2000)

return;

        lastScannedRef.current = code;

        lastTimeRef.current = now;

        await handleCodeScan(code, currentVenteId);

      },

      () => {}

    );

  } catch {

    flash("✖ Webcam inaccessible !");

    setScanning(false);

  }

}, 300);

};

```

5. Gestion du projet avec Git

Le projet LibrairyPro a été versionné avec Git et hébergé sur GitHub (dépôt public : github.com/Doha-it/PFE). Le dépôt est organisé en deux dossiers principaux : PFE-main (code source complet de l'application React + Spring Boot) et gestion-produits (module de gestion des produits), accompagnés du fichier BDD.sql contenant le script de création et d'initialisation de la base de données MySQL.

Le workflow Git adopté consiste en des commits réguliers par fonctionnalité développée, permettant de tracer l'historique de développement et de revenir à un état antérieur en cas de régression. Les commits ont été réalisés sur la branche principale (main), chaque commit correspondant à l'ajout ou la correction d'une fonctionnalité spécifique.

6. Conclusion du chapitre

Ce chapitre a présenté la réalisation concrète du projet **LibrairyPro**, en détaillant les technologies utilisées ainsi que les principales fonctionnalités implémentées.

Le système développé permet une gestion complète et moderne d'un point de vente de type librairie, intégrant la lecture de codes-barres, la gestion des stocks, la facturation automatique et l'analyse des ventes en temps réel.

CHAPITRE 5 TESTS ET ÉVALUATION



1. Introduction du chapitre

Ce chapitre présente la phase de tests et d'évaluation du système **LibrairyPro**. L'objectif est de vérifier le bon fonctionnement des différentes fonctionnalités développées, d'identifier les éventuelles anomalies et de mesurer la conformité du système par rapport aux objectifs initiaux du projet.

2. Objectifs des tests

Les tests réalisés ont pour objectifs de :

- Vérifier la stabilité de l'application
- Tester les fonctionnalités principales (CRUD, vente, stock)
- Valider le système de code-barres
- Contrôler la génération des factures PDF
- Tester la sécurité et l'authentification JWT
- Évaluer la performance globale du système

3. Méthodologie de test

Nous avons adopté une approche de tests **manuels fonctionnels**, basée sur des scénarios d'utilisation réels simulant le comportement d'un administrateur et d'un caissier.

Les tests sont organisés selon les modules du système :

- Tests positifs (flux nominaux) : vérification du comportement attendu dans des conditions normales d'utilisation.
- Tests négatifs (cas d'erreur) : vérification du comportement du système en cas de saisies invalides, d'accès non autorisés, ou de données manquantes.

4. Scénarios de test



Tableau des tests fonctionnels

ID	Fonctionnalité	Entrée	Résultat attendu	Résultat obtenu	Statut
T01	Login utilisateur	email + mot de passe	Accès dashboard	Accès accordé avec token JWT	✓ OK
T02	Ajout produit	Formulaire produit	Produit ajouté en base	Produit visible dans stock	✓ OK
T03	Scan code-barres	Code barre	Ajout produit panier	Produit ajouté automatiquement	✓ OK
T04	Modification stock	Changement quantité	Stock mis à jour	Mise à jour instantanée	✓ OK
T05	Création vente	Panier validé	Facture générée	PDF téléchargé correctement	✓ OK
T06	Gestion caissier	CRUD utilisateur	Caissier créé/modifié	Données mises à jour	✓ OK
T07	Dashboard KPI	Données vente	Graphiques mis à jour	Données temps réel affichées	✓ OK

Table 4 Tests fonctionnels [positifs (flux nominaux)]

ID	Module	Entrée / Action	Résultat attendu	Résultat obtenu	Statut
TN01	Authentification	Mot de passe incorrect	Message d'erreur 401 Unauthorized	Email ou mot de passe incorrect !	✓ OK
TN02	Authentification	Forme incorrecte	Erreur de validation login	Please include an '@' In the email address.	✓ OK
TN03	Ajout caissier	Numéro de téléphone incomplète	Erreur de validation formulaire	Le numéro doit contenir exactement 10 chiffres	✓ OK
TN04	Code-barres	Code-barres inexistant en base	Message 'Produit non trouvé'	Produit introuvable !	✓ OK
TN05	Gestion produits	Ajout produit sans code-barres	Erreur de validation formulaire	Please fill out this field	✓ OK
TN06	Ajout produit	Code-barres déjà existant (doublon)	Erreur de contrainte UNIQUE	Ce code-barres correspond déjà à ...	✓ OK

Table 5 Tests fonctionnels [négatifs (cas d'erreur)]

5. Métriques de performance

Opération testée	Temps mesuré	Seuil acceptable	Résultat
Chargement initial de l'application	< 2s	< 3s	✓ Satisfaisant
Identification produit par code-barres (API)	< 300ms	< 1s	✓ Très bon
Chargement du dashboard avec graphiques	< 1.5s	< 3s	✓ Satisfaisant
Génération et affichage de la facture PDF	< 500ms	< 2s	✓ Très bon
Chargement liste produits (50 items)	< 800ms	< 2s	✓ Satisfaisant
Validation d'une vente (API POST)	< 400ms	< 1s	✓ Très bon

Table 6 5. Métriques de performance

6. Limites identifiées

Malgré les résultats positifs, certaines limites existent :

- Absence de scanner physique natif (version web limitée)
- Dépendance à la connexion internet
- Interface mobile pas encore en développement
- Pas encore de mode offline

7. Conclusion du chapitre

Ce chapitre a permis de valider le bon fonctionnement du système **LibrairyPro** à travers différents scénarios de tests. Les résultats obtenus confirment que les objectifs du projet ont été globalement atteints, avec une solution stable, performante et adaptée aux besoins d'un point de vente moderne.

CHAPITRE 6 BILAN ET PERSPECTIVES



1. Introduction du chapitre

Ce dernier chapitre présente une synthèse globale du projet **LibrairyPro**. Il met en évidence les objectifs atteints, les compétences acquises durant le développement, ainsi que les perspectives d'amélioration et d'évolution du système.

2. Synthèse du projet

Le projet LibrairyPro est une application web de gestion de point de vente destinée à une librairie, intégrant un système de lecture de codes-barres, une gestion complète des produits, des ventes, des stocks et des utilisateurs. L'application repose sur une architecture moderne en trois couches : Frontend React + Vite, Backend Spring Boot (API REST sécurisée JWT), et Base de données relationnelle MySQL.

3. Bilan des objectifs

3.1 Objectifs pleinement atteints

- Développement d'un système POS complet avec interfaces administrateur et caissier.
- Gestion des produits (CRUD) avec code-barres unique (EAN-13/ISBN) et suivi des stocks en temps réel.
- Intégration de la lecture de codes-barres dans le flux de vente.
- Génération automatique de factures PDF.
- Mise en place d'un dashboard analytique avec KPIs et graphiques mensuels.
- Sécurisation des accès avec authentification JWT et gestion des rôles ADMIN/CAISSIER.

3.2 Objectifs partiellement atteints

- Scan de codes-barres via caméra : la fonctionnalité nécessite un scanner USB externe ou une saisie clavier ; le scan via caméra de smartphone n'est pas encore implémenté.
- Tests de charge : des tests fonctionnels ont été réalisés, mais les tests de performance sous charge simultanée (plusieurs caissiers actifs) n'ont pas encore été effectués.

3.3 Objectifs non atteints

- Interface mobile : une version responsive partielle est disponible, mais l'expérience sur mobile reste sous-optimale par rapport à une application native ou PWA.
- Mode hors ligne (Offline POS) : prévu mais non implémenté dans cette version en raison des contraintes de temps. Il nécessite l'utilisation de Service Workers et d'une solution de stockage local (IndexedDB).
- Système de messagerie interne fonctionnel entre administrateur et caissiers.

4. Compétences développées

Ce projet a permis de renforcer plusieurs compétences techniques et organisationnelles :

Compétences techniques :

- Développement full stack (React / Spring Boot)
- Conception d'API REST
- Gestion de bases de données relationnelles
- Authentification sécurisée (JWT)
- Manipulation de flux de données en temps réel

Compétences méthodologiques :

- Analyse des besoins
- Architecture logicielle
- Gestion de projet
- Organisation en modules

5. Difficultés rencontrées

Durant le développement, plusieurs défis ont été rencontrés :

- Synchronisation entre frontend et backend
- Implémentation du système de code-barres
- Optimisation des performances du dashboard

Ces difficultés ont été surmontées grâce à une approche progressive et modulaire.

6. Perspectives d'amélioration

Afin d'améliorer le système **LibrairyPro**, plusieurs évolutions sont envisagées :

1. Application mobile (React Native)

- Intégration du scan de code-barres via caméra
- Remplacement des saisies manuelles
- Utilisation de [expo-barcode-scanner](#)

2. Mode hors ligne (Offline POS)

- Synchronisation automatique
- Stockage locale temporaire
- Reprise des ventes sans internet

3. Intelligence artificielle

- Prédiction des ventes
- Analyse des produits les plus rentables
- Détection des ruptures de stock

4. Extension multi-boutiques

- Gestion de plusieurs librairies
- Tableau de bord global
- Comparaison des performances

5. Amélioration UX/UI

- Interface plus optimisée mobile
- Animation plus fluide
- Accessibilité améliorée

7. Conclusion du chapitre

Ce chapitre a permis de dresser un bilan global du projet **LibrairyPro**. Les résultats obtenus montrent que le système répond efficacement aux besoins d'un point de vente moderne grâce à une architecture robuste et des fonctionnalités avancées.

Ce projet constitue une base solide pour de futures améliorations, notamment vers des solutions mobiles et intelligentes.

CONCLUSION GÉNÉRALE

Le projet **LibrairyPro** représente une solution complète de gestion de point de vente pour une librairie, intégrant des technologies modernes telles que React, Spring Boot et MySQL.

L'objectif principal était de concevoir un système capable d'automatiser les opérations de vente, de gestion de stock et d'analyse des performances. Cet objectif a été atteint avec succès grâce à une architecture bien structurée et une implémentation progressive.

Ce projet nous a permis de développer des compétences techniques avancées en développement full stack ainsi qu'une meilleure compréhension des systèmes d'information modernes.

Enfin, LibrairyPro ouvre la voie à plusieurs améliorations futures, notamment l'intégration mobile et l'intelligence artificielle, ce qui confirme son potentiel d'évolution vers un système encore plus intelligent et autonome.

BIBLIOGRAPHIE

-[1] K. C. Laudon et J. P. Laudon, *Management Information Systems*, Pearson Education, 2020.

-[2] GS1 International, *GS1 General Specifications*, 2022.

Disponible : <https://www.gs1.org/standards/barcodes>

-[3] J. T. Mentzer et al., *Defining Supply Chain Management*, Journal of Business Logistics, 2001.

-[4] M. Jones, J. Bradley et N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, 2015.

Disponible : <https://www.rfc-editor.org/rfc/rfc7519>

-[5] Retail Dive Research, *The Evolution of POS Technology*, Retail Dive.

Disponible : <https://www.retaildive.com>

-[6] Block Inc., *Square Point of Sale Documentation*, 2024.

Disponible : <https://developer.squareup.com>

-[7] Shopify Inc., *Shopify POS Documentation*, 2024.

Disponible : <https://help.shopify.com/en/manual/sell-in-person>

-[8] Lightspeed Commerce Inc., *Lightspeed Retail POS Documentation*, 2024 .

Disponible : <https://support.lightspeedhq.com>

ANNEXE

Annexe A : Script SQL de création de la base de données (extrait)

Le script complet de création et d'initialisation de la base de données MySQL est disponible dans le dépôt GitHub du projet (fichier BDD.sql).

L'extrait ci-dessous présente la structure des tables principales :

```
-- Annexe A : Extrait BDD.sql - Création des tables principales

-- =====

CREATE TABLE utilisateurs (

  id            INT            AUTO_INCREMENT PRIMARY KEY,

  nom           VARCHAR(100)   NOT NULL,

  telephone     VARCHAR(20)    NULL,

  email         VARCHAR(150)   NOT NULL UNIQUE,

  mot_de_passe  VARCHAR(255)   NOT NULL,

  role          ENUM('admin', 'caissier') NOT NULL DEFAULT 'caissier',

  created_at    TIMESTAMP      DEFAULT CURRENT_TIMESTAMP -- Enregistre la date de
création du compte.

);

-- =====

CREATE TABLE fournisseurs (

  id            INT            AUTO_INCREMENT PRIMARY KEY,

  nom           VARCHAR(100)   NOT NULL,

  telephone     VARCHAR(20)    NULL,

  email         VARCHAR(150)   NULL

);
```

```
-- =====

CREATE TABLE produits (

  id            INT            AUTO_INCREMENT PRIMARY KEY,

  fournisseur_id INT,

  categorie_id   BIGINT,

  nom            VARCHAR(200)   NOT NULL,

  code_barres    VARCHAR(100)   NOT NULL UNIQUE,  -- scannerCodeBarres()

  prix           DECIMAL(10, 2) NOT NULL,

  quantite       INT            NOT NULL DEFAULT 0,

  CONSTRAINT fk_produit_fournisseur

    FOREIGN KEY (fournisseur_id) REFERENCES fournisseurs(id)

    ON DELETE SET NULL,

  CONSTRAINT fk_produit_categorie

    FOREIGN KEY (categorie_id)   REFERENCES categories(id)

    ON DELETE SET NULL

);

-- =====

CREATE TABLE ventes (

  id            INT            AUTO_INCREMENT PRIMARY KEY,

  utilisateur_id INT            NOT NULL,          -- caissier qui effectue la vente

  date          DATETIME       DEFAULT CURRENT_TIMESTAMP,

  total         DECIMAL(10, 2) NOT NULL DEFAULT 0,

  statut        ENUM('Abandonnées', 'validee', 'annulee') NOT NULL DEFAULT

  'Abandonnées',

  CONSTRAINT fk_vente_utilisateur

    FOREIGN KEY (utilisateur_id) REFERENCES utilisateurs(id)

);
```


Annexe B : Captures de quelques erreurs

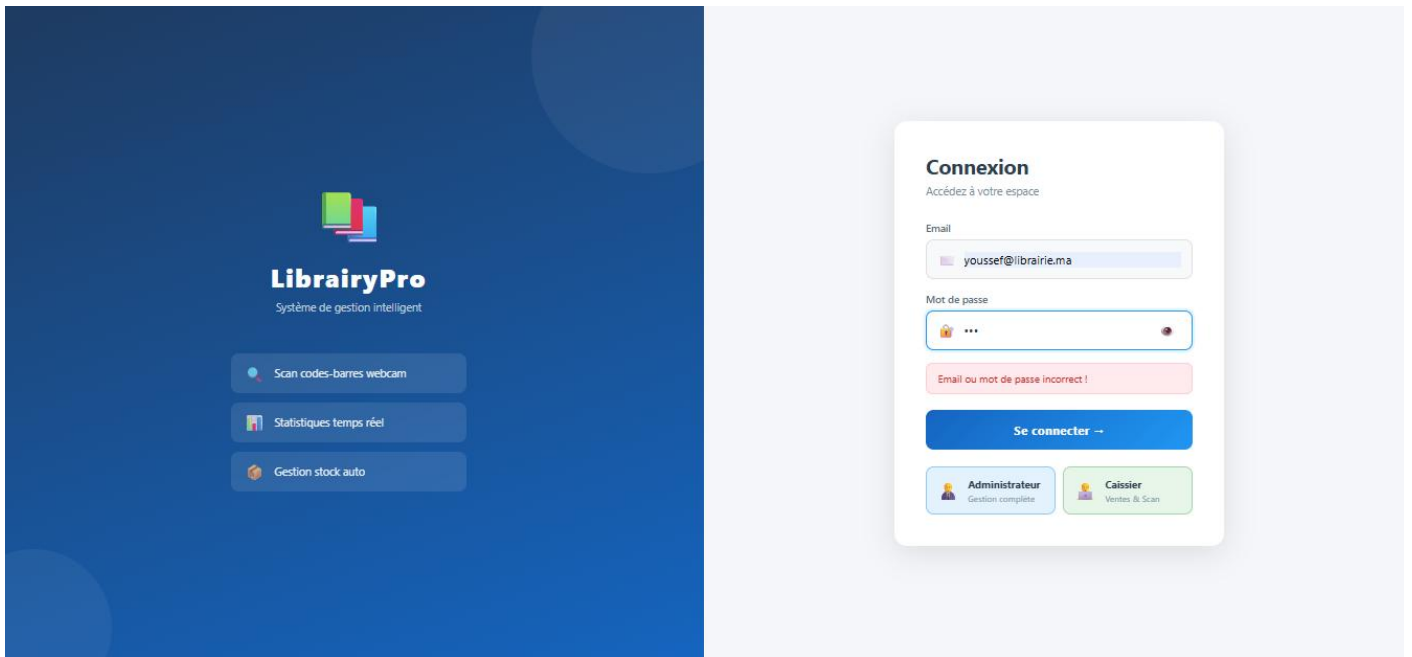


Figure Erreur 1 Mot de passe incorrect

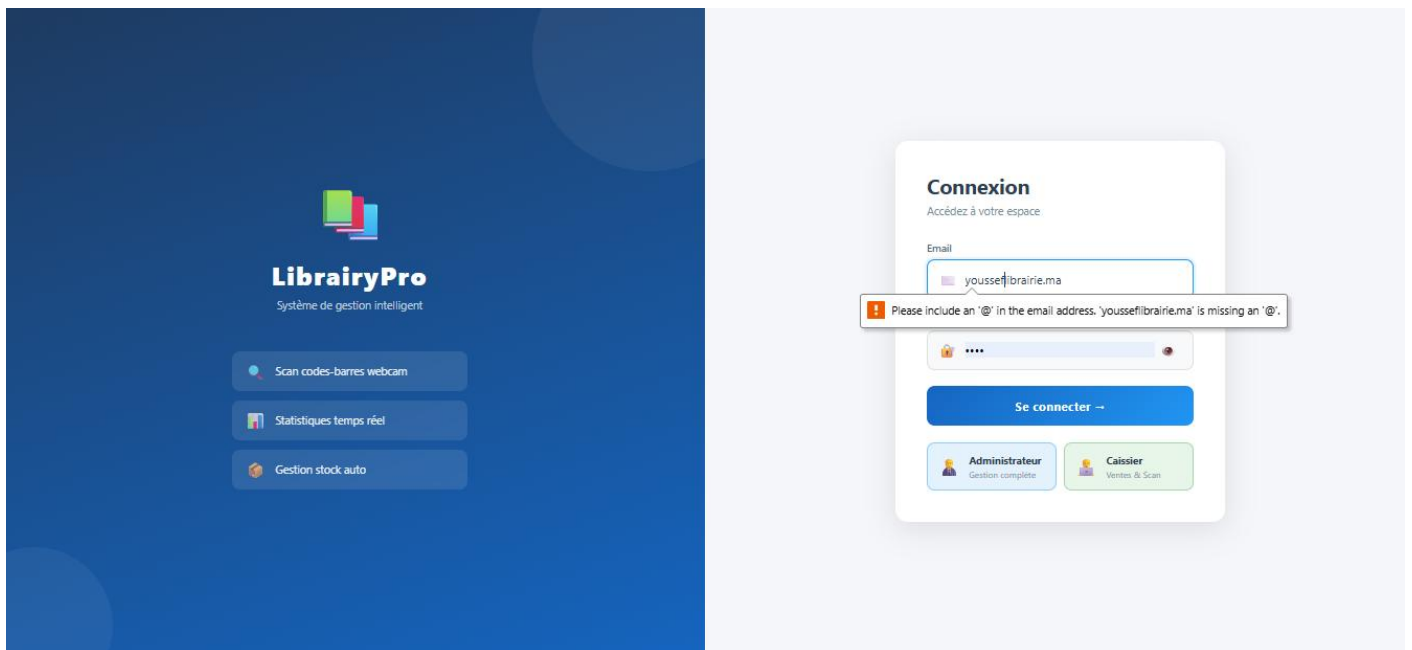


Figure Erreur 2 Fausse Forme d'e-mail

LibrairyPro

Accueil

Stock/Produit

Fournisseur

Caissier

Vente

Historique

Paramètre

D Doha

Administrateur

Déconnexion

Stock & Produits

75 produits enregistrés

X Fermer

Tous (75)

Livres Scolaires (17)

Fournitures Scolaires (30)

Informatique (4)

Littérature (4)

Sciences (5)

Langues (3)

Arts (5)

Papeterie (5)

Tous fournisseurs

Hachette Maroc (12)

Hamelin Maroc (14)

Papeterie Scolaire Maroc (27)

BIC & Fournitures Maroc (10)

Maison des Arts et Culture (5)

Éditions Scientifiques Modernes (5)

+ Nouveau produit

Nom du produit *

Le rouge et le noir

Code-barres *

5498263547824

Prix (DH) *

0.00

Quantité *

56

Catégorie

Littérature

Fournisseur

Hamelin Maroc

Ajouter

Annuler

Please fill out this field.

Recherche...

Nom : A-Z Z-A

Prix : ↑ ↓

Stock : ↑ ↓

NOM	CODE-BARRES	CATÉGORIE	PRIX	QUANTITÉ	FOURNISSEUR	ACTIONS
Mathématiques 1ère Bac	9782891722856	Livres Scolaires	85 DH	49	Hachette Maroc	

Figure Erreur 3 Insertion incomplète

LibrairyPro

Caissier

Youssef

Scanner

Panier (0)

Facture

Produit introuvable !

Catalogue produits

Rechercher un produit...

Nom

↑

NOM	CODE-BARRES	CATÉGORIE	PRIX	STOCK	ACTION
A la première personne	9782747535755	Littérature	100.00 DH	29	+ Ajouter
Agenda scolaire 2024-2025	6111245789028	Fournitures Scolaires	25.00 DH	77	+ Ajouter
Agrafeuse de bureau	6111245789032	Papeterie	35.00 DH	25	+ Ajouter
Algorithmique et Programmation	9782891789012	Informatique	95.00 DH	28	+ Ajouter
Anglais 1ère Bac	9782891456789	Livres Scolaires	70.00 DH	54	+ Ajouter
Anglais Tronc Commun	9782891456790	Livres Scolaires	65.00 DH	59	+ Ajouter
Architecture Contemporaine	9782180800026	Arts	240.00 DH	9	+ Ajouter
Art Moderne	9782180800023	Arts	210.00 DH	12	+ Ajouter
Atlas du Corps Humain	97821808000201	Sciences	110.00 DH	18	+ Ajouter
Boîte 1000 agrafes	6111245789033	Papeterie	10.00 DH	97	+ Ajouter

Scanner un produit

Vente #32 — 0 article(s) — 0.00 DH

Caméra inactive

Démarrer le scan

Saisir un code-barres...

+ Ajouter

Figure Erreur 4 Produit introuvable [code-barre incorrecte]

Stock & Produits
75 produits enregistrés

Tous (75) Livres Scolaires (17) Fournitures Scolaires (30) Informatique (4) Littérature (4) Sciences (6) Langues (3) Arts (6) Papeterie (5)

Tous fournisseurs Hachette Maroc (12) Hamelin Maroc (14) Papeterie Scolaire Maroc (27) BIC & Fournitures Maroc (10) Maison des Arts et Culture (6) Éditions Scientifiques Modernes (6)

+ Nouveau produit

Nom du produit * Ex: Mathématiques 3ème

Code-barres * 9782747535755

▲ Ce code-barres correspond déjà à la première personne

Prix (DH) * 0.00

Quantité * 0

Catégorie -- Choisir catégorie --

Fournisseur -- Aucun --

Ajouter Annuler

Recherche... Nom : A-Z Z-A Prix : 1 1 Stock : 1 1

Figure Erreur 6 Produit déjà existant [au cours d'ajout d'un produit par l'admin]

Fournisseurs
6 fournisseurs enregistrés

Modifier le fournisseur

Nom * Hachette Maroc

Téléphone 061234567

Le numéro doit contenir exactement 10 chiffres

Email contact@hachette.ma

Modifier Annuler

H Hachette Maroc
0612345678
contact@hachette.ma
Modifier Supprimer

H Hamelin Maroc
0623456789
hamelin@papeterie.ma
Modifier Supprimer

P Papeterie Scolaire Maroc
0634567890
contact@papeterie.ma
Modifier Supprimer

B BIC & Fournitures Maroc
0645678901
contact@bicmaroc.ma
Modifier Supprimer

Figure Erreur 5 Forme de téléphone non valide

Annexe C : Lien du dépôt GitHub

Dépôt public du projet LibrairyPro : <https://github.com/Doha-it/PFE> . Le dépôt contient :

- Le code source complet (frontend React + backend Spring Boot),
- Le script SQL de la base de données (BDD.sql),
- La documentation du projet.